

高效使用創進一號主機(CPU叢集)計算資源

方育斌、周朝宜、陳威宇

教材：
https://github.com/robinfang7/nchc_hpc_tutorial
https://github.com/waue0920/nchc_hpc_slurm_example

Agenda

1. 國網中心的超級電腦

1. CPU叢集系統 (創進一號、台灣杉三號)
2. iService計算資源服務網

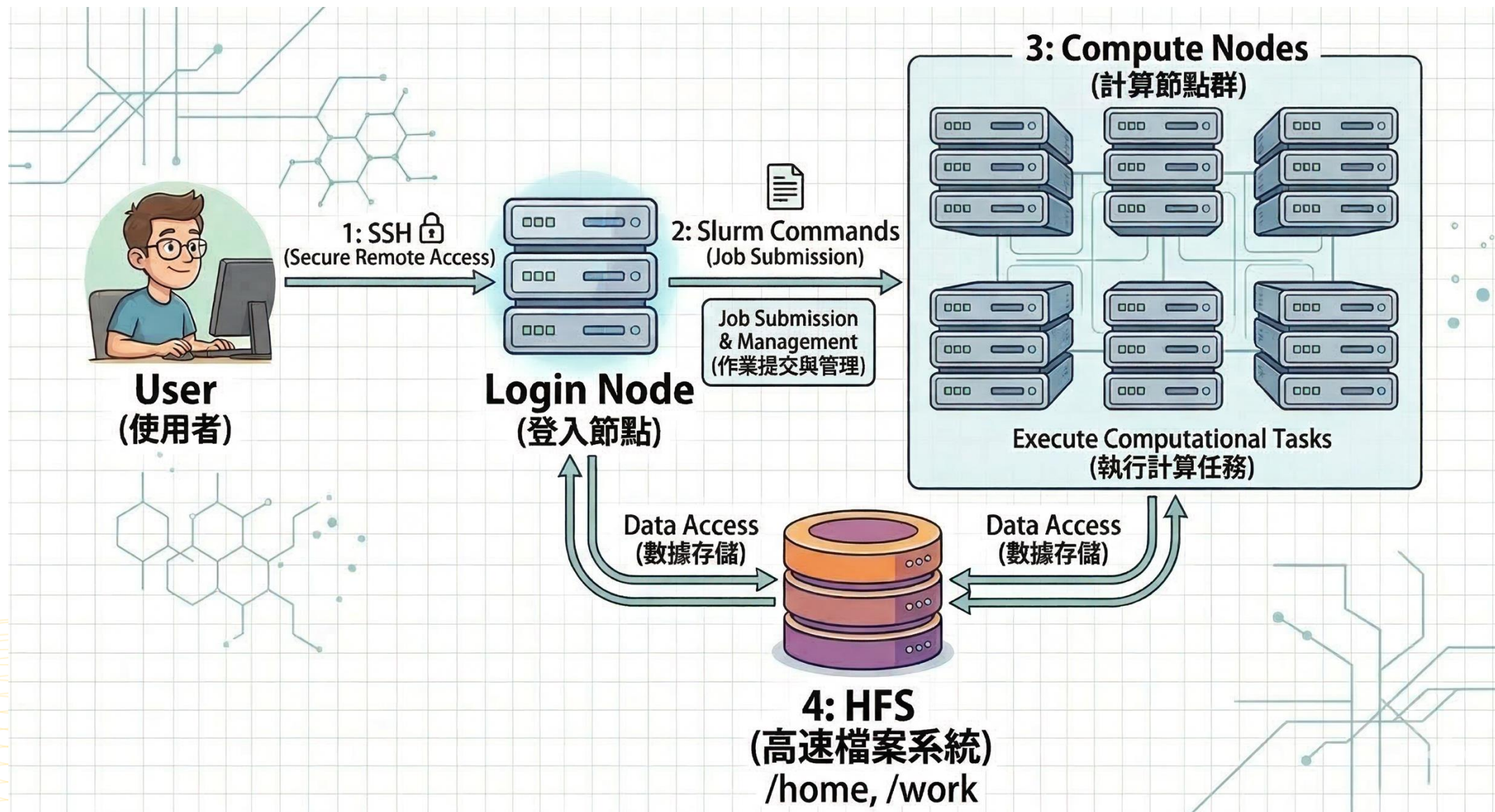
2. 軟體

1. Slurm任務調度
2. Lmod軟體管理
3. Singularity/Apptainer容器
4. 安裝軟體

3. 實作展示

1. MPI benchmark
2. miniWeather on x86
3. miniWeather on arm

4. 問題與討論



創進一號Forerunner 1

- 創進一號 (Forerunner I) 為國網中心最新建置的高速計算主機，具備558台計算伺服器，共可提供62,496個計算核心；每台計算伺服器配備2顆56核心Intel Xeon Platinum 8480+ 2.0GHz處理器、InfiniBand 200Gb/s高速網路、512GB記憶體。實際整體最佳效能約為3.5PFlops。
- 創進一號 (Forerunner I) 為泛用型計算主機，可廣泛提供各領域如物理、化學、數學、大氣科學、工程應用、生命科學等研究所需，適合產官學研各用戶使用，更鼓勵有大計算需求之研究多加運用。除專業套裝軟體外，系統亦提供數種編譯器與MPI函式庫供研究人員自行開發計算程式。



台灣杉三號 (TAIWANIA 3)

- 台灣杉三號(Taiwania 3)為國內提供開放服務申請的最大CPU高速計算主機(2021年)，擁有900個計算節點。台灣杉三號全系統共計 50,400 個計算核心(Cores)、與台灣杉二號共用平行檔案系統總容量約為 9.4PB，搭配InfiniBand HDR100 100Gbps高速網路，整體效能可達2.7千兆次浮點運算(PFLOPS)。
- 本系統為泛用型計算主機，可提供多種領域的計算服務，包含物理、化學、數學、大氣科學、工程應用、生命科學等。除商業套裝軟體外，系統亦提供數種編譯器與本MPI函式庫供研究人員自行開發計算程式之用。



台灣杉三號 (TAIWANIA 3)

- 2020/11
- 計算力 2.7 PFLOPS
- Top500 # 181
- Green500 # 69

創進一號、台灣杉三號規格比較

	創進一號	台灣杉三號
總算力	3.5 PFLOPS	2.7 PFLOPS
節點數	558	900
檔案系統總容量	9.3 PB	9.4 PB
節點CPU	2顆Intel Xeon Platinum 8480+ 2.0GHz	2顆 Intel® Xeon® Platinum 8280 2.4GHz
節點CPU核心	112	56
節點記憶體容量	512GB	192GB
網路頻寬	InfiniBand NDR200 200Gb/s	InfiniBand HDR100 100Gbps
費率	0.1/0.2/0.2/0.37	0.08/0.08/0.16/0.16

四種CPU小時費率為國科會計畫、學術計畫、政府與法人計畫、企業與個人計畫

系統架構與計算資源

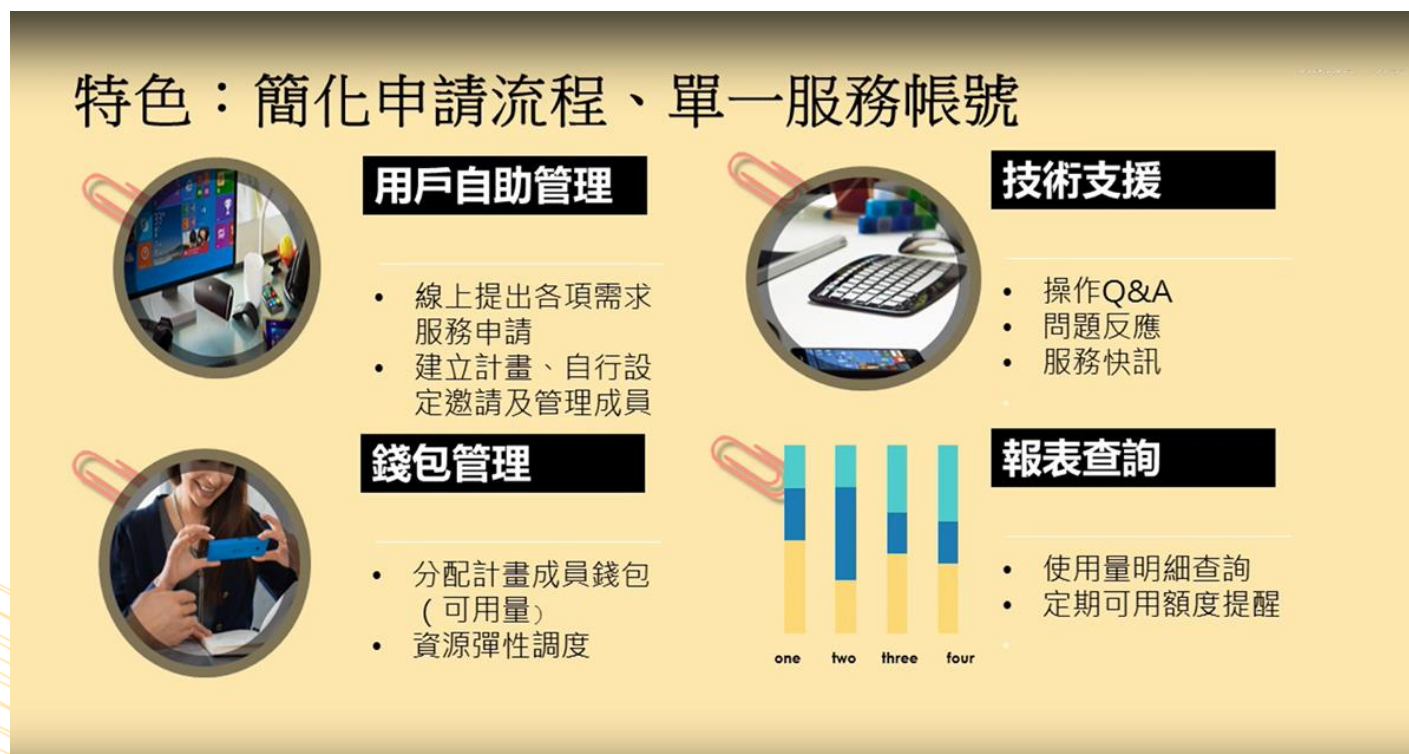
創進一號共有 552台x86計算節點與6台x86繪圖節點 (總計1116個處理器、62,496個核心、12個3D繪圖用A40 GPU) 與40台ARM計算節點 (總計80個處理器、5,760個核心)。

- 558 台x86計算節點由雙 CPU 插槽組成，每個插槽具為 Intel Xeon Platinum 8480+, 56cores, 2.0GHz。
- 40台ARM計算節點由雙 CPU 插槽組成，每個插槽為NVIDIA Grace CPU Superchip 使用Arm Neoverse V2核心，內建72核心，2.6GHz。

節點種類	總個數 (節點)	單位計算資源 (節點)				
		CPU 插槽數	CPU核心數	記憶體 (GB)	Nvidia A40	InfiniBand NDR 200Gb/s
X86	552	2	112	512	-	1
X86繪圖節點	6	2	112	512	2	1
ARM	40	2	144	240	-	1

整體效能最高達3.5 Petaflops，9.2 PB儲存容量。

- <https://iservice.nchc.org.tw/>
- 提供國網中心各服務平台的帳戶申請及管理，包含計畫成員管理、預算和經費管理、使用資源報表查詢、平台操作技術支援等功能，其中計算資源申請服務，如創進一號、臺灣AI雲(TWCC)等，透過iService計算資源服務網提供服務。



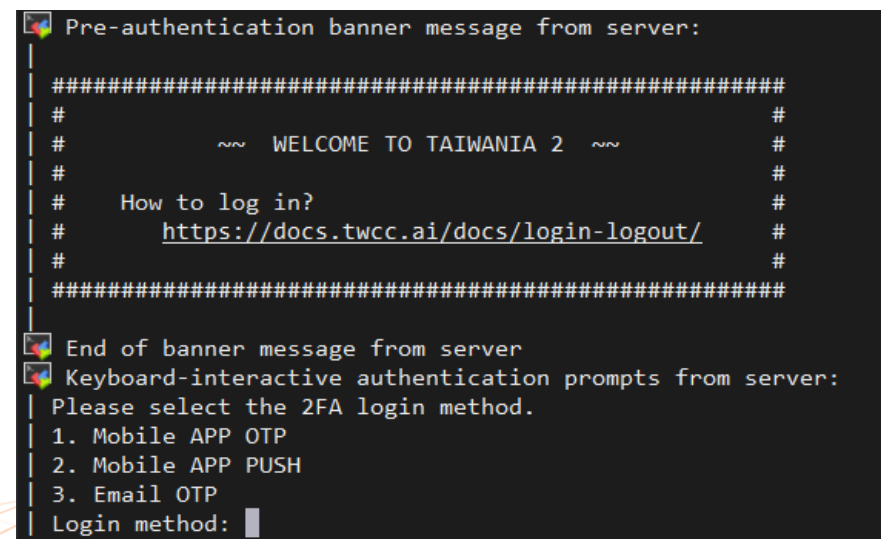
帳號有兩種

■ iService會員帳號

- 登入iService網站、TWCC網站
- 帳號的格式是電子信箱

■ 主機帳號

- 登入HPC主機，創進一號、晶創主機、Taiwania 2



主機帳號與OTP設定

修改主機帳號基本資料

主機帳號

主機帳號

██████ (啟用)

主機密碼

變更主機密碼

OTP 載具: 有效

刪除使用者載具

:::最近一次變更主機密碼日期: 2025-02-04
09:19:16

最近一次成功刪除載具日期:

注意: 已發送APP 載具註冊通知信件

使用 IDExpert 手機 APP(APP OTP 或 Push)進行登入節點雙因子(2FA) 驗證作業前, 請至 ████████ 信箱收取載具註冊通知信件, 並依內容指示完成 APP 載具註冊作業。

▶ Email OTP 電子郵件: ████████

說明:

1. 登入節點雙因子(2FA) 驗證作業方式, 選項說明:

1). Mobile APP OTP: 需安裝APP, 並完成註冊作業。開啟 APP 後取得 OTP 碼以完成 2FA 驗證。

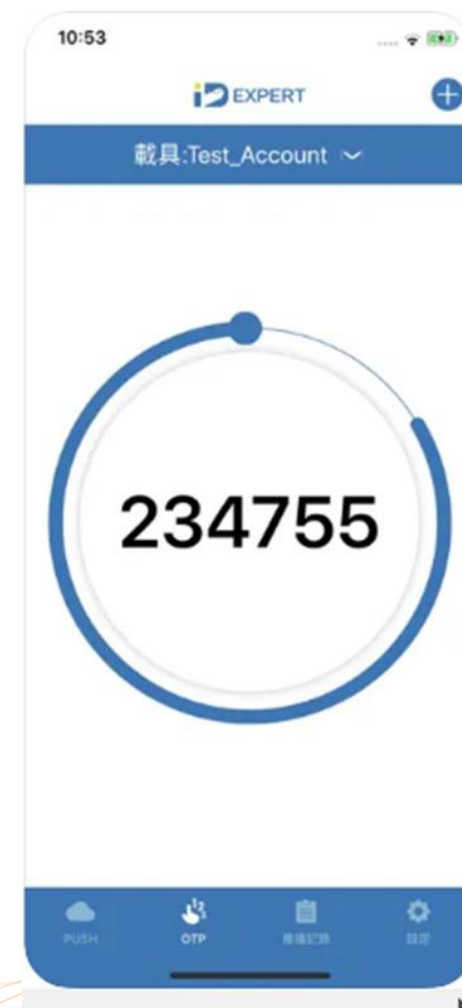
2). Mobile APP PUSH: 需安裝 APP, 並完成註冊作業。由 login node 觸發推播驗證機制至用戶的手機以完成 2FA 驗證。

3). Email OTP: 將透過您註冊於 iService E-mail 取得 OTP 驗證碼以完成 2FA 驗證。

2. 註冊與登入作業, 請參考線上說明文件。

3. 用戶更換手機需重新安裝 APP 或原安裝的 APP 刪除後重新安裝, 請參考線上說明文件第3點, 重建載具。

4. 透過 APP QRCode 進行註冊服務, 每位帳號僅可註冊一支手機。



- https://iservice.nchc.org.tw/module_page.php?module=nchc_service#nchc_service/nchc_service.php?action=nchc_motp_unix_account_edit_v3
- https://iservice.nchc.org.tw/nchc_service/nchc_service_qa_single.php?qa_code=774

高速檔案系統 HFS Hyper File System

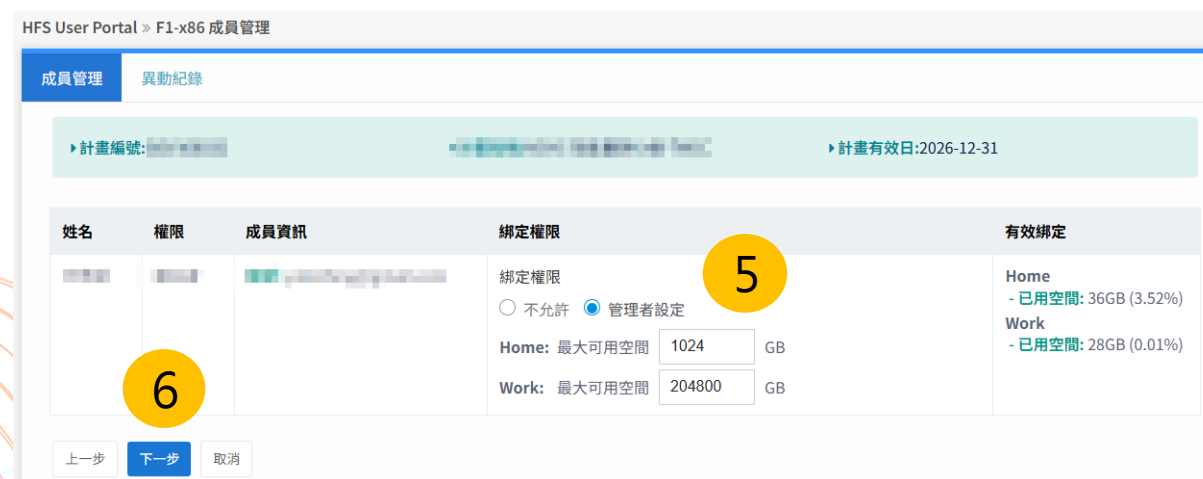
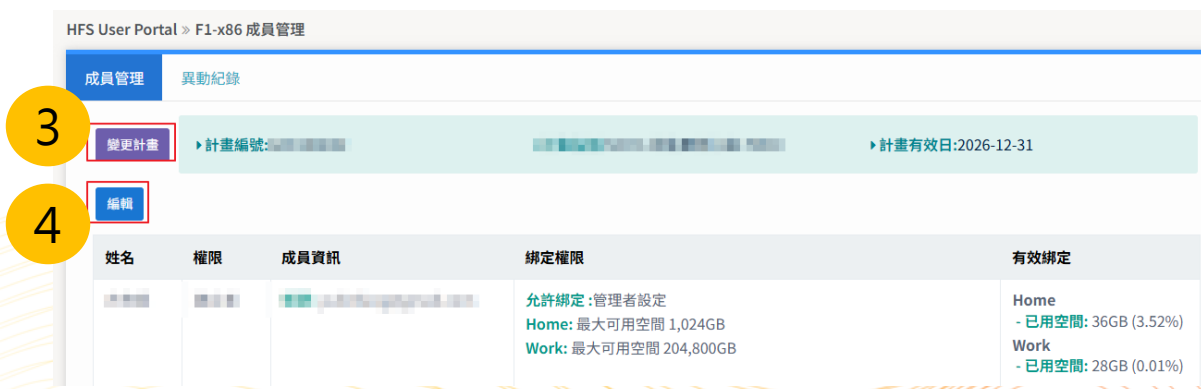
	創進一號	台灣杉二號/台灣杉三號
免費額度	/home 100 GB /work 100 GB (國科會計畫1536GB)	/home 100 GB /work 100 GB (國科會計畫1536GB)
最高容量	/home 1024 GB (1TB) /work 204800 GB (200TB)	/home 10240 GB (10TB) /work 204800 GB (200TB)
Hostname	f1-ilgn01.nchc.org.tw	xdata1.twcc.ai
Port	port 22	port 22

收費方法：每日計費，每天根據前一日的用量平均值進行扣款，以更實際地反應用戶的使用情況。
HFS儲存空間：(*Home、Work與Project 費率相同) (單位：元)

計畫類型	每日 1 GB
國科會計畫	0.0069
學術計畫	0.035
政府與法人計畫	0.07
企業與個人計畫	0.14

設定與查詢HFS 容量

計劃建立者和管理者有權限設定HFS容量



登入節點

■ SSH Tools

- ▣ MobaXterm, Putty for Windows
- ▣ Terminal for Mac and Linux

■ 雙因子(2FA) 驗證

	Host name	Port
創進一號 x86	f1-ilgn01.nchc.org.tw	22
創進一號 arm	f1-nlgn01.nchc.org.tw	22
台灣杉三號	twnia3.nchc.org.tw	22

The screenshot shows the MobaXterm interface. On the left, a sidebar lists various sessions, including 'Nano5'. The main terminal window displays the following text:

```
Keyboard-interactive authentication prompts from server:
Please select the 2FA login method.
1. Mobile APP OTP
2. Mobile APP PUSH
3. Email OTP
Login method: 2
Password:
End of keyboard-interactive prompts from server

? MobaXterm Personal Edition v23.3 ?
(SSH client, X server and network tools)

[?] SSH session to u8880716@nano5.nchc.org.tw
? Direct SSH : ✓
? SSH compression : ✓
? SSH-browser : ✓
? X11-forwarding : ✓ (remote display is forwarded through SSH)

[?] For more info, ctrl+click on help or visit our website.

Please check your push token.
[PASS] The push verification succeeded.
```

The screenshot shows the terminal output of the SSH session. It displays a pre-authentication banner and prompts for 2FA login method and password.

```
Pre-authentication banner message from server:

#####
#                                     #
#           ~~ WELCOME TO TAIWANIA 2 ~~           #
#                                     #
#   How to log in?                                     #
#   https://docs.twcc.ai/docs/login-logout/           #
#                                     #
#####

End of banner message from server
Keyboard-interactive authentication prompts from server:
Please select the 2FA login method.
1. Mobile APP OTP
2. Mobile APP PUSH
3. Email OTP
Login method: 2
Password:
```

資料傳輸節點

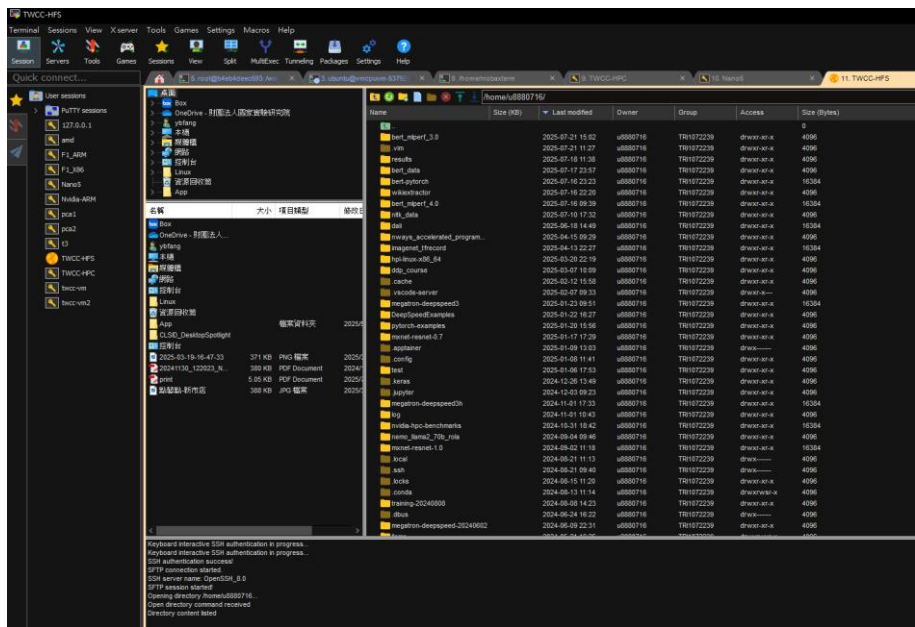
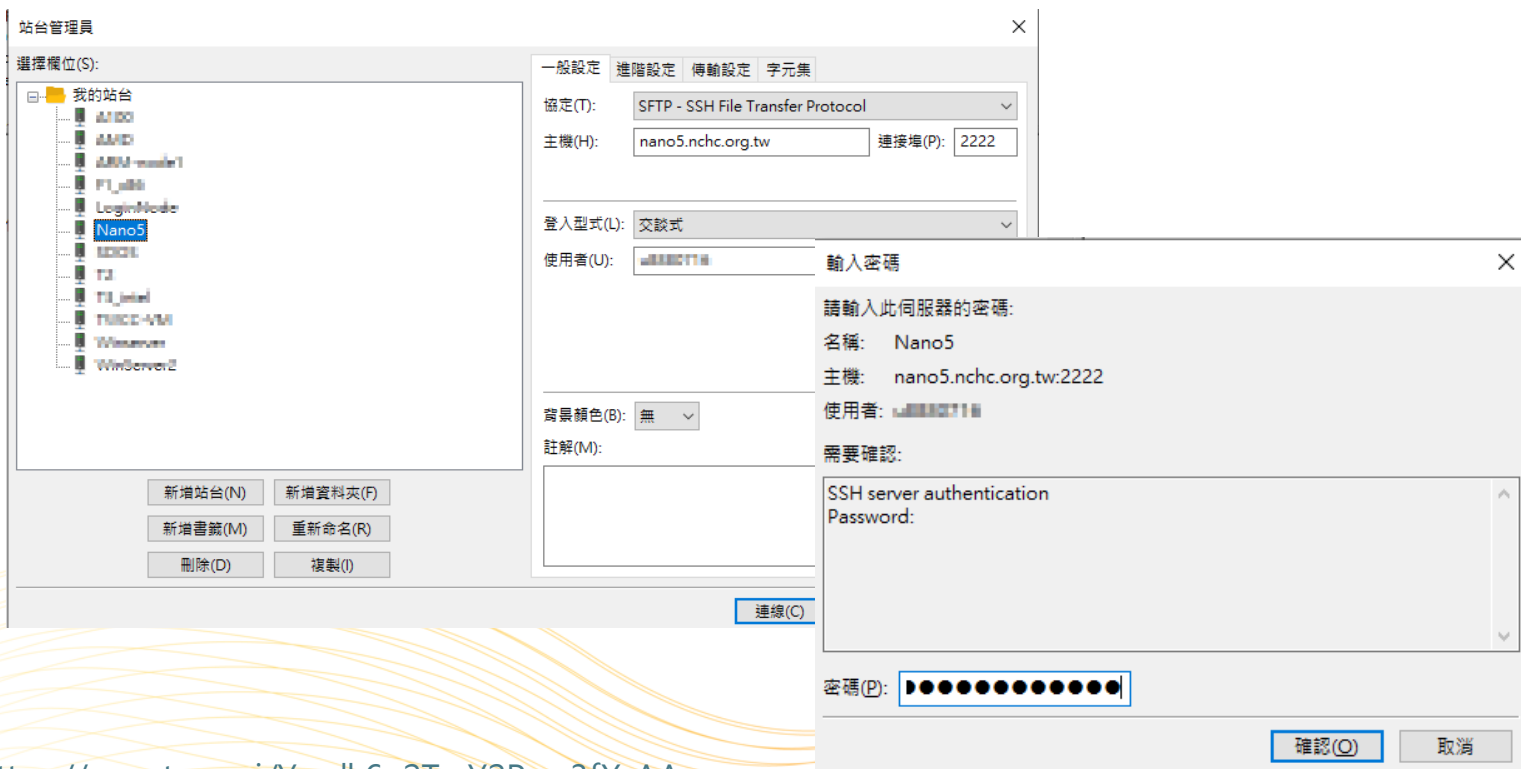


■ SFTP Tools

- Filezilla, WinSCP, MobaXterm
- sftp command

■ 雙因子(2FA) 驗證

	Host name	Port
創進一號 x86	f1-dtn01.nchc.org.tw	22
創進一號 arm	f1-nlgn01.nchc.org.tw	22
台灣杉三號	xdata1.twcc.ai	22



https://man.twcc.ai/Yg_dk6n2T_-Y2Pmx3fXzAA
<https://man.twcc.ai/@twccdocs/doc-hfs-main-zh/%2F%40twccdocs%2Fhfs-overview-zh>

互動式繪圖節點

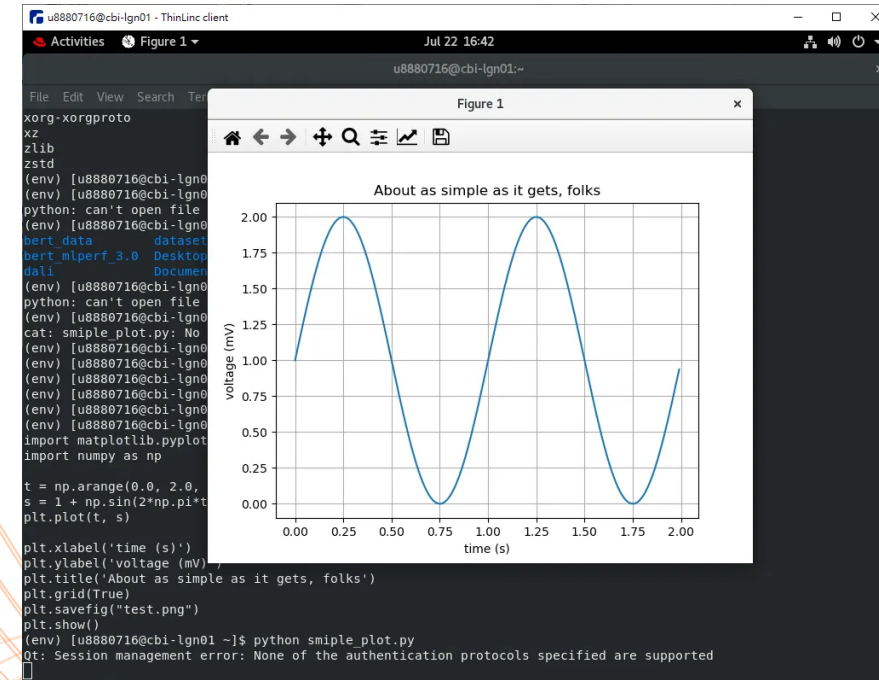
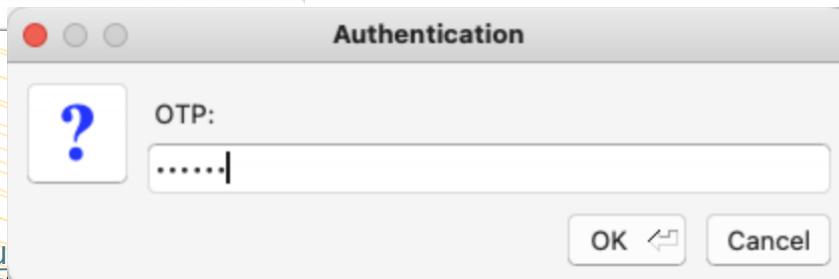
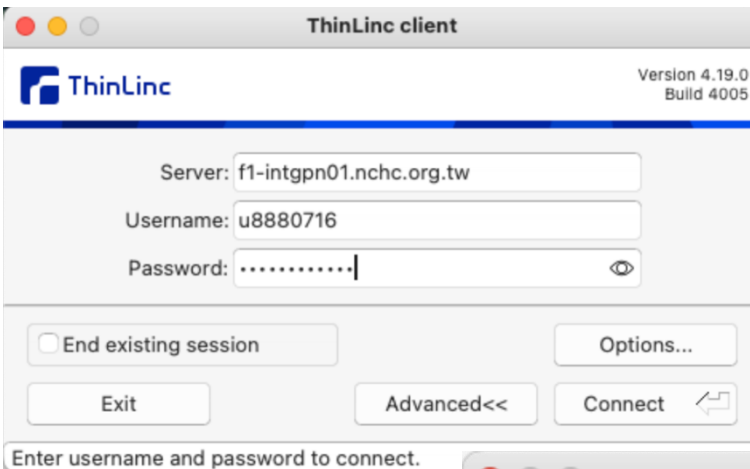
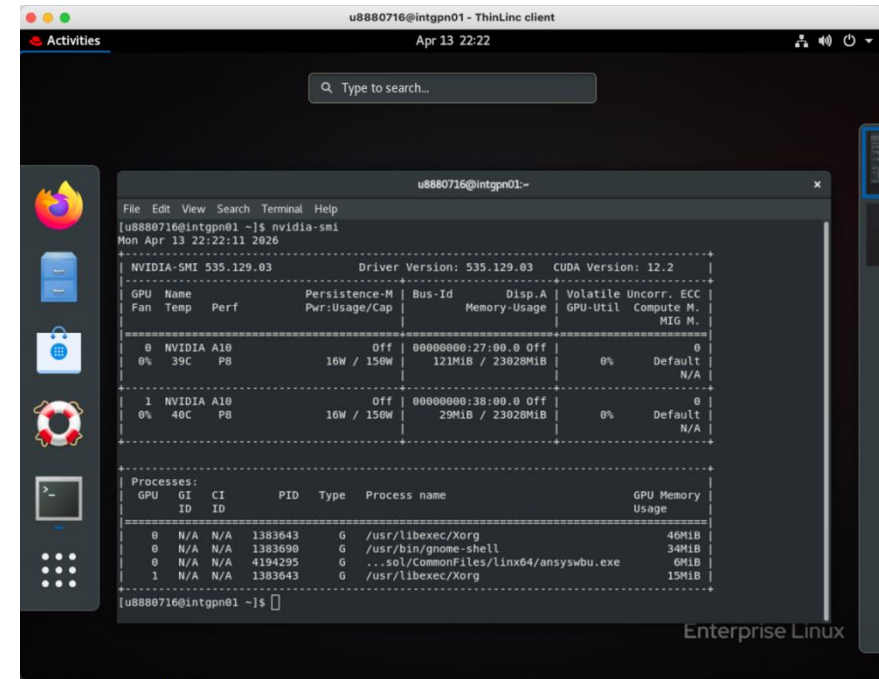
■ Tools

- ThinLinc for Windows, Mac, and Linux

- MobaXterm for Windows

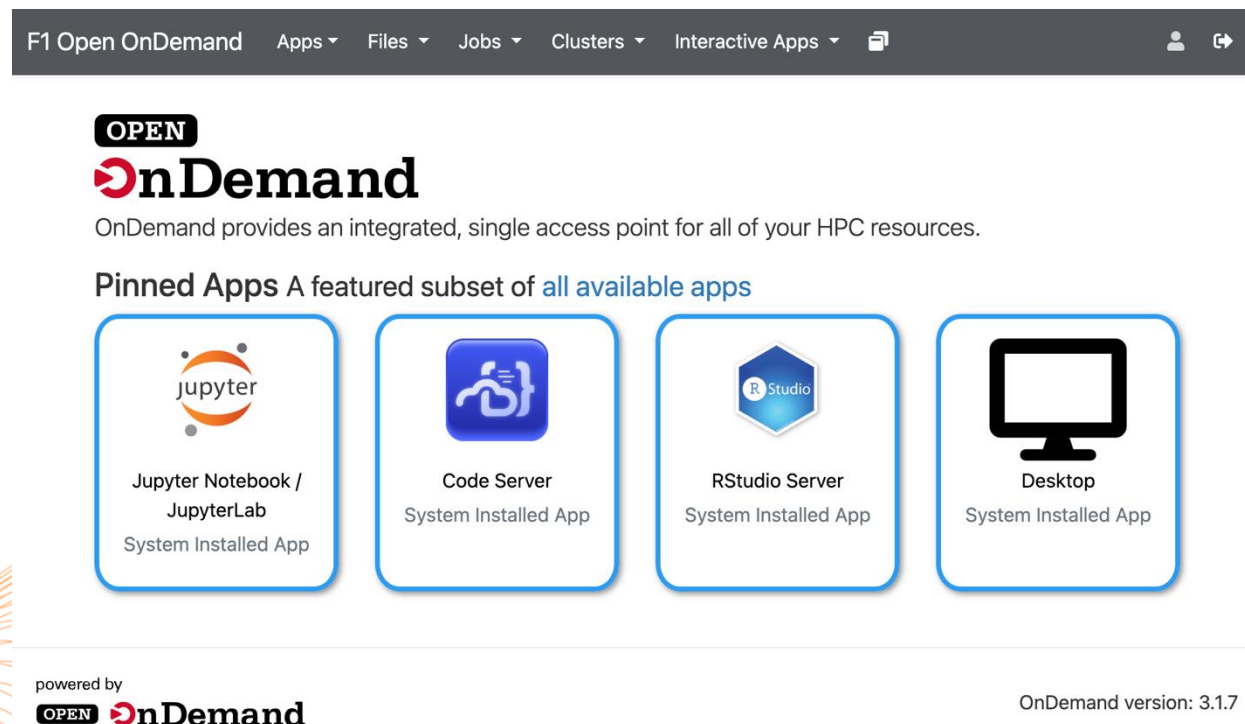
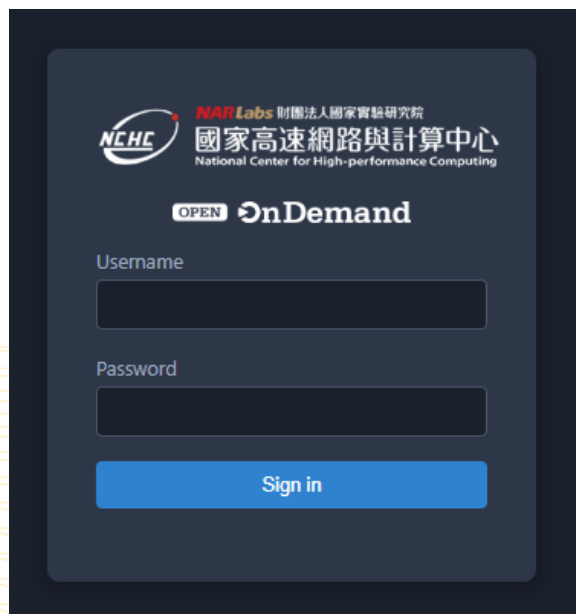
■ Host name: f1-intgpn01.nchc.org.tw

■ Port: 1111



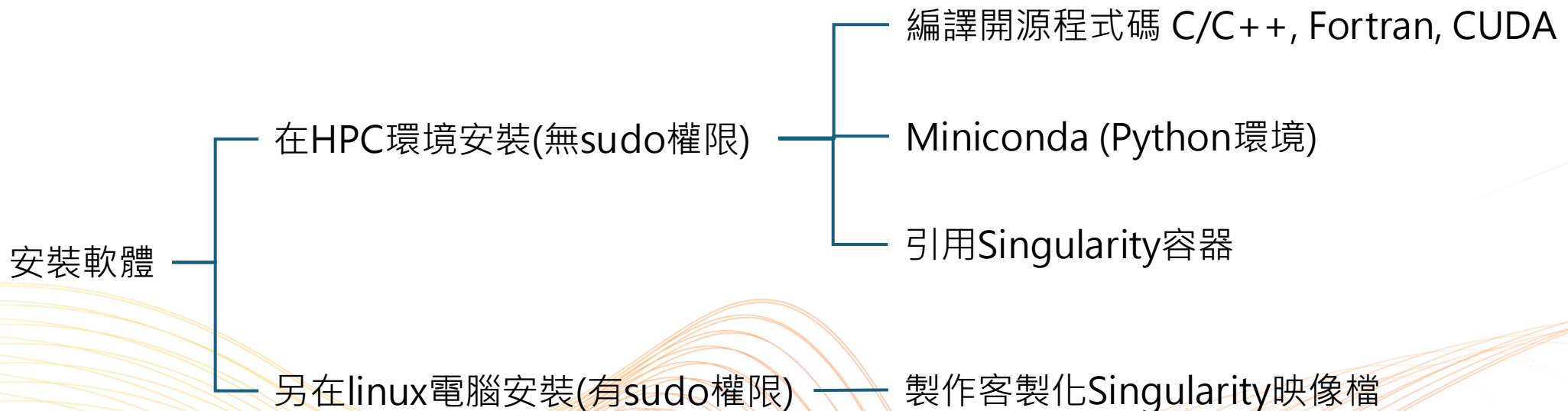
Open OnDemand

- Open OnDemand 是一個開源的 HPC 叢集網頁介面，專為簡化使用者操作而設計。
- 其提供以下功能
 - 檔案管理：使用類似於檔案總管的圖形化介面，輕鬆瀏覽、上傳與下載檔案。
 - 終端機存取：在網頁上直接啟動終端機，無需 SSH 工具，便於操作。
 - 應用程式支援：能在網頁上直接啟動常用的程式編輯器，我們提供Code Server、Jupyter Notebook 以及 R Studio。
- <https://f1-stn01.nchc.org.tw/>



HPC Cluster Software

- Slurm任務調度工具，取得、查詢計算資源、提交工作。
- Lmod環境管理工具，引用已安裝的編譯器、函式庫、應用程式。
- Singularity/Apptainer容器，引用已封裝依賴項與應用程式的獨立環境。
- 安裝軟體



Slurm (Simple Linux Utility for Resource Management)

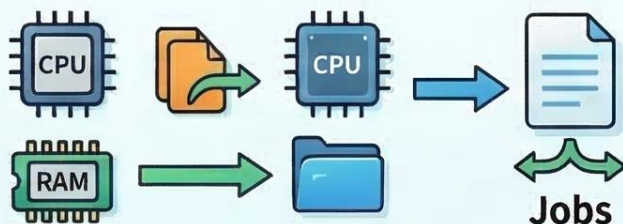
極簡Linux資源管理工具

用於超級電腦和叢集的自由開源任務調度



資源分配 (Resource Allocation)

- 動態分配：按照用戶的需求 (節點、CPU核數、記憶體)
- 支援獨佔或非獨佔模式
- 保證資源有效利用



作業排程與調度 (Job Scheduling)

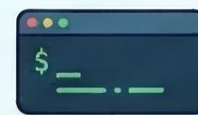
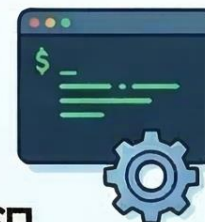


- 根據優先順序、作業依賴性、資源狀態將作業分配至合適節點
- 支援多種排程策略 (FIFO, Priority, Preemption)



作業管理與監控 (Job Management & Monitoring)

- 用戶使用命令管理和監控作業
 - sbatch 提交批次
 - srun 執行互動
 - squeue 查詢狀態
 - scancel 取消
- 追蹤作業資源使用情況
- 透過記帳功能

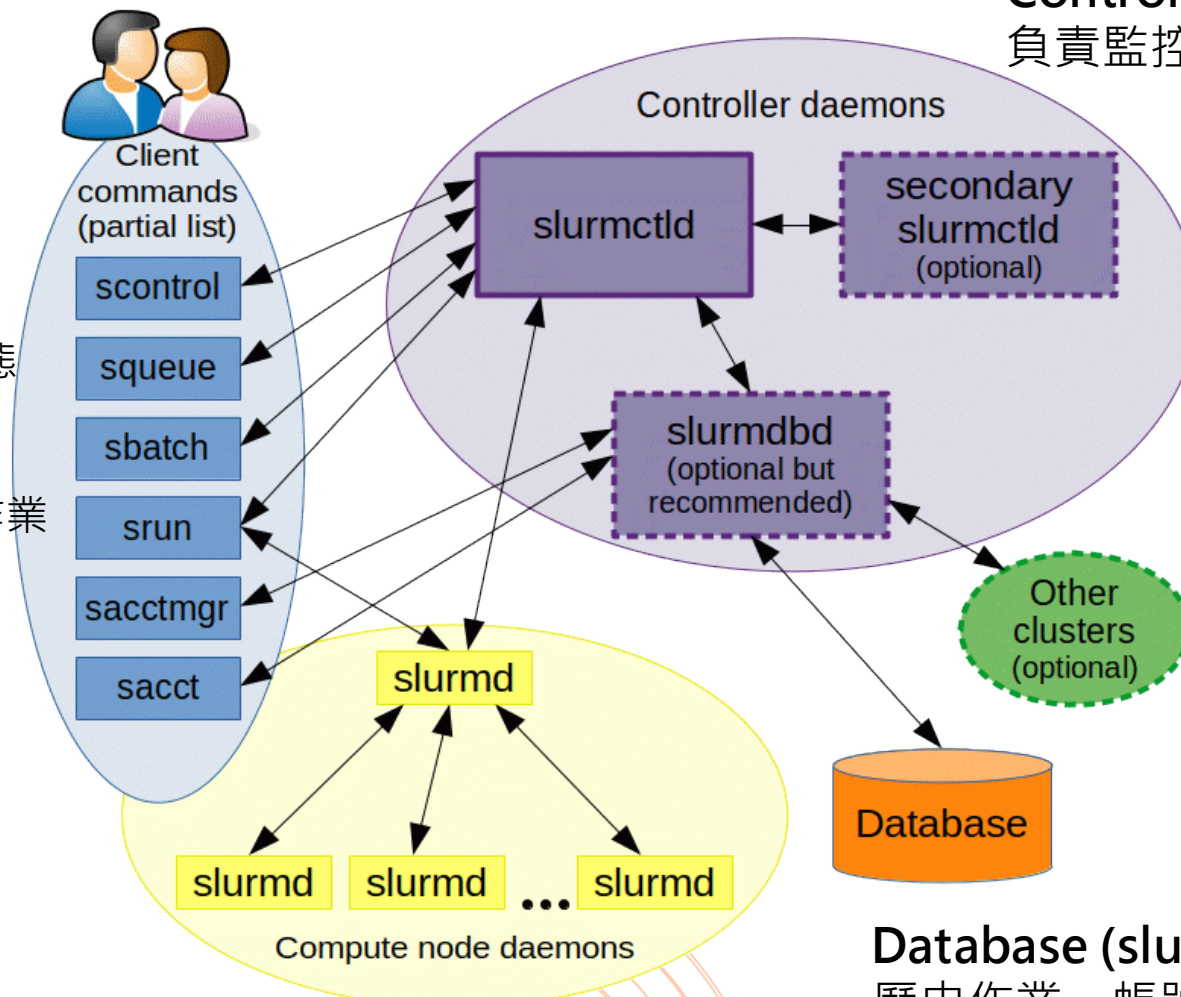


Top500
超級電腦廣泛採用



Slurm architecture

Controller (slurmctld)：叢集的總管，負責監控資源狀態與調度作業。



用戶常用的指令：

`sinfo`: 查詢目前分區與計算節點狀態

`scontrol`: 查詢分區與計算節點組態

`squeue`: 查詢目前job狀態

`sbatch <slurm script>` 提交運算作業

`salloc`: 互動式提交運算作業

`srun`: 互動式提交運算作業

`sacct`: 查詢過去的作业資訊

Compute Daemon (slurmd)：安裝在每一台計算節點上的小幫手，負責執行總管交代的任務。

Database (slurmdbd)：記錄所有歷史作業、帳號配額與帳單資訊。

Slurm entities

Node 計算節點

- 叢集中的伺服器。這些是實際執行使用者作業的硬體資源。

Partition分區

- 分區將叢集中的節點按照用途、硬體配置或管理政策劃分為不同群組，類似於工作隊列。
- 用戶在提交作業時需要指定目標分區，每個分區可以設定不同的資源限制與排程政策。

Job 作業

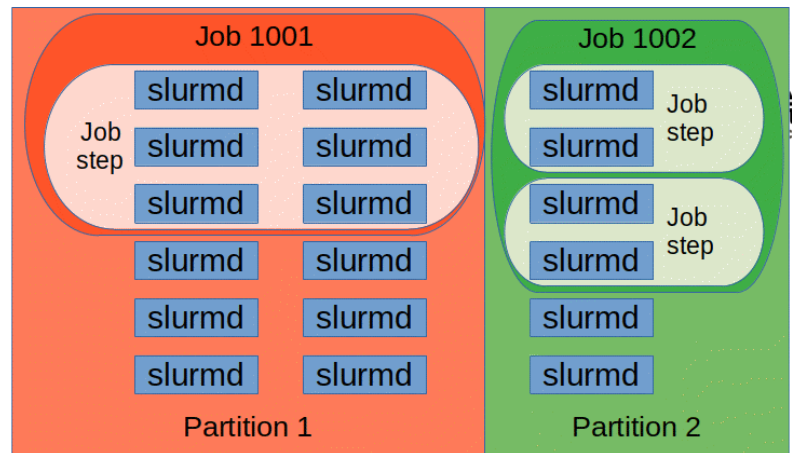
- 用戶提交的計算任務，記錄了資源需求、作業狀態、提交時間、執行時間、輸出檔案等資訊。
- 每個作業都有一個唯一的作業識別碼 (Job ID)，系統根據該識別碼追蹤作業進度。

Job steps

- 作業可以進一步細分為一個或多個作業步驟，每個步驟通常對應一次 `srun` 命令的執行。
- 這允許同一個作業內部進行序列或平行計算，並分別監控每個步驟的狀態與資源使用情形。

Tasks

- 一個job step可以包含一個或多個tasks，每個task代表一個要在計算節點上執行的工作。
- 在平行計算中，tasks通常用於執行同一個任務的多個實例，從而加速計算的進行。



Partition: 創進一號

X86架構CPU計算環境

佇列名稱	可用核心數範圍	最長可執行時間 (hour)	每位用戶可同時執行工作數	每位用戶可排隊工作數	計算工作派送節點
vscode	1~112	2	1	1	限制從 Open OnDemand Portal派送，即 stn[01-02]
jupyter	1~112	2	1	1	
rstudio	1~112	2	1	1	
development	1~1120	8	1	1	限制從登入節點、互動式繪圖節點派送，即 ilgn[01-02] 與 intgpn[01-02]
ct112	1~112	96	5	10	
ct448	113~448	96	4	8	
ct1k	449~1120	64 (更新前為96)	3	6	
ct2k	1121~2240	48	2	4	
ct4k	2241~4480	48	1	2	
ct8k	4481~8960	48	1	1	
visual-dev	1~112	2	1	1	限制從互動式繪圖節點派送，即 intgpn[01-02]
visual	1~448	48	1	1	

ARM架構CPU計算環境

佇列名稱	可用核心數範圍	最長可執行時間 (hour)	每位用戶可同時執行工作數	每位用戶可排隊工作數	計算工作派送節點
arm-dev	1~1440	2	1	1	限制從登入節點派送，即 ilgn[01-02]
arm144	1~144	48	6	6	
arm576	145~576	48	4	4	
arm1440	577~1440	48	2	2	

<https://man.twcc.ai/@f1-manual/partition>

查看Partition的狀態

sinfo -s

```
[u8880716@ilgn01 ~]$ sinfo -s
```

PARTITION	AVAIL	TIMELIMIT	NODES(A/I/O/T)	NODELIST
vscode	up	2:00:00	0/5/1/6	gpn[01-06]
jupyter	up	2:00:00	0/5/1/6	gpn[01-06]
rstudio	up	2:00:00	0/5/1/6	gpn[01-06]
desktop	up	2:00:00	0/5/1/6	gpn[01-06]
development	up	8:00:00	84/65/1/150	icpnq[101-156,201-256,301-310,321-348]
ct112	up	4-00:00:00	274/59/3/336	icpnq[101-156,201-256,301-356,401-456,501-556,601-656]
ct448	up	4-00:00:00	274/59/3/336	icpnq[101-156,201-256,301-356,401-456,501-556,601-656]
ct1k	up	2-16:00:00	323/64/3/390	icpnq[101-156,201-256,301-356,401-456,501-556,601-656,701-754]
ct2k	up	2-00:00:00	323/64/3/390	icpnq[101-156,201-256,301-356,401-456,501-556,601-656,701-754]
ct4k	up	2-00:00:00	323/64/3/390	icpnq[101-156,201-256,301-356,401-456,501-556,601-656,701-754]
ct8k	up	2-00:00:00	323/64/3/390	icpnq[101-156,201-256,301-356,401-456,501-556,601-656,701-754]
cf112	up	4-00:00:00	84/37/1/122	icpnq[101-156,201-256,301-310]
cf448	up	4-00:00:00	84/37/1/122	icpnq[101-156,201-256,301-310]
cf1k	up	2-16:00:00	84/37/1/122	icpnq[101-156,201-256,301-310]
cf2k	up	2-00:00:00	84/37/1/122	icpnq[101-156,201-256,301-310]
cf4k	up	2-00:00:00	84/37/1/122	icpnq[101-156,201-256,301-310]
hm112	up	4-00:00:00	2/8/0/10	icpnq[311-320]
hm448	up	4-00:00:00	2/8/0/10	icpnq[311-320]
visual-dev	up	2:00:00	0/5/1/6	gpn[01-06]
visual	up	2-00:00:00	0/3/1/4	gpn[03-06]
arm-dev	up	2:00:00	9/30/1/40	ncpn[01-40]
arm144	up	2-00:00:00	9/30/1/40	ncpn[01-40]
arm576	up	2-00:00:00	0/29/1/30	ncpn[11-40]
arm1440	up	2-00:00:00	0/29/1/30	ncpn[11-40]
debug	up	infinite	0/2/0/2	icpnq[755-756]

A : alloc, mixed 整台完全/部分使用

I : idle 整台閒置

O : drain 維修中

T : total nodes 節點數量

執行方式

■ 腳本

- sbatch <slurm script>

■ 互動式

- srun：以一行指令取得資源與執行運算
- salloc：先取得資源，進入計算節點，在命令列上逐一執行運算指令，最後關閉job

Slurm script

run.slurm

索取資源

引用模組

引用容器

執行運算

```
#!/bin/bash
#SBATCH --account=<PROJECT_ID>      # (-A) iService Project ID
#SBATCH --job-name=test              # (-J) Job name
#SBATCH --partition=development      # (-p) Slurm partition
#SBATCH --nodes=2                    # (-N) Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=4            # (-c) Number of cores per MPI task
#SBATCH --ntasks-per-node=1          # Maximum number of tasks on each node
#SBATCH --time=0-00:30:00            # (-t) Wall time limit (days-hrs:min:sec)
#SBATCH --output=%x-%j.out           # (-o) Path to the standard output file, %x is job name, %j is job id
#SBATCH --error=%x-%j.err            # (-e) Path to the standard error file
#SBATCH --mail-type=END,FAIL         # Mail events (NONE, BEGIN, END, FAIL, ALL)
#SBATCH --mail-user=user@example.com # Where to send mail. Set this to your email address

module purge
module load intel/....

SIF="/path/openmpi.sif"
SINGULARITY="singularity run -B /work:/work --nv $SIF"

srun --mpi=pmix -n 4 $SINGULARITY ./hello
mpiexec ./hello
```

Submit and manager job

提交Job Script

- sbatch run.slurm

查看使用者的job 狀態

- squeue -u <username>
- squeue -j <job_id>

查看正在運行的Job 組態

- scontrol show job <job_id>

刪除 job

- scancel <job_id>

srun

On Forerunner1

```
srun -A <project> -p development --ntasks-per-node=1 <command>
```

```
[u8880716@cbi-lgn01 ~]$ srun -A GOV108032 -J test -p dev --gres=gpu:1 nvidia-smi
Fri Jul 25 15:15:02 2025
```

NVIDIA-SMI 550.127.08				Driver Version: 550.127.08				CUDA Version: 12.4			
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Memory-Usage	Volatile	Uncorr.	ECC		
Fan	Temp		Pwr:Usage/Cap				GPU-Util	Compute	M.		
								MIG	M.		
0	NVIDIA H100	80GB HBM3	On	00000000:86:00:0	Off		0%	0			
N/A	26C	P0	70W / 700W	1MiB / 81559MiB				Default	Disabled		

```
Processes:
GPU  GI  CI  PID  Type  Process name  GPU Memory
   ID  ID               Usage
=====
No running processes found
```

salloc

```
salloc -A <project> -p gtest --nodes 1 --ntasks-per-node 1 --gres=gpu:1 # 索取資源
echo $SLURM_NODELIST # check allocated compute node
ssh <compute_node> # 進入計算節點
<command1> ... # 執行運算
<command2> ...
scancel <job_ID> # 結束工作
```

```
[u8880716@un-ln01 ~]$ salloc -A GOV108032 -p gtest --nodes 1 --ntasks-per-node 1 --gres=gpu:1
WARNING: When login node is DOWN, Your Job will FAIL!
WARNING: Please use `sbatch` instead :)
salloc: Granted job allocation 753642
salloc: Waiting for resource configuration
salloc: Nodes gn1002 are ready for job
[u8880716@un-ln01 ~]$ echo $SLURM_NODELIST
gn1002
[u8880716@un-ln01 ~]$ scancel 753642
salloc: Job allocation 753642 has been revoked.
[u8880716@un-ln01 ~]$ queue -u u8880716
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
-------	-----------	------	------	----	------	-------	------------------

Slurm環境變數

執行job運算，產生Slurm的環境變數，可以做為平行運算的參數

以下是常用的Slurm環境變數

變數名稱	描述
SLURM_JOB_ID	Job ID
SLURM_JOB_NAME	Job name
SLURM_JOB_ACCOUNT	Project id
SLURM_JOB_NUM_NODES SLURM_NNODES	Number of nodes allocated to job
SLURM_JOB_NODELIST SLURM_NODELIST	Nodes assigned to job
SLURM_NTASKS_PER_NODE	Number of tasks requested per node.
SLURM_NTASKS	Total tasks in job
SLURM_GPUS_ON_NODE	Number of GPUs per node. (Only on Nano5)

列出所有Slurm環境變數

```
srun -A <ProjectID> -p normal --gres=gpu:2 printenv | grep SLURM
```

```
srun: job 81496 queued and waiting for resources
srun: job 81496 has been allocated resources
SLURM_CONF=/etc/slurm/slurm.conf
SLURM_PRIO_PROCESS=0
SLURM_UMASK=0022
SLURM_CLUSTER_NAME=hpc
SLURM_SUBMIT_DIR=/home/u8880716/llama_factory_b842457e
SLURM_SUBMIT_HOST=cbi-lgn01
SLURM_JOB_NAME=printenv
SLURM_JOB_CPUS_PER_NODE=1
SLURM_MEM_PER_NODE=409600
SLURM_NTASKS=1
SLURM_NPROCS=1
SLURM_DISTRIBUTION=cyclic,pack
SLURMD_DEBUG=2
SLURM_JOB_ID=81496
SLURM_JOBID=81496
SLURM_STEP_ID=0
SLURM_STEPID=0
SLURM_NNODES=1
SLURM_JOB_NUM_NODES=1
SLURM_NODELIST=hgpn03
SLURM_JOB_PARTITION=normal
SLURM_TASKS_PER_NODE=1
SLURM_JOB_UID=12409
SLURM_JOB_USER=u8880716
SLURM_JOB_GID=18361
SLURM_JOB_GROUP=TRI1072239
```

```
SLURM_JOB_ACCOUNT=gov108032
SLURM_JOB_QOS=normal
SLURM_JOB_NODELIST=hgpn03
SLURM_STEP_NODELIST=hgpn03
SLURM_STEP_NUM_NODES=1
SLURM_STEP_NUM_TASKS=1
SLURM_STEP_TASKS_PER_NODE=1
SLURM_STEP_LAUNCHER_PORT=43051
SLURM_SRUN_COMM_PORT=43051
SLURM_SRUN_COMM_HOST=172.21.101.1
SLURM_GPUS_ON_NODE=2
SLURM_STEP_GPUS=0,4
SLURM_TOPOLOGY_ADDR=ibsw1.hgpn03
SLURM_TOPOLOGY_ADDR_PATTERN=switch.node
SLURM_CPUS_ON_NODE=1
SLURM_CPU_BIND=quiet,mask_cpu:0x000000000000001000000000000000
SLURM_CPU_BIND_LIST=0x000000000000001000000000000000
SLURM_CPU_BIND_TYPE=mask_cpu:
SLURM_CPU_BIND_VERBOSE=quiet
SLURM_JOB_END_TIME=1766045527
SLURM_JOB_START_TIME=1765872727
SLURM_TASK_PID=2708572
SLURM_NODEID=0
SLURM_PROCID=0
SLURM_LOCALID=0
SLURM_LAUNCH_NODE_IPADDR=172.21.101.1
SLURM_GTIDS=0
SLURMD_NODENAME=hgpn03
```

高效能運算 (HPC) Lmod 環境管理 | 探索並優化軟體模組

解決 HPC 環境中的軟體管理複雜性

1 HPC 系統面臨的挑戰

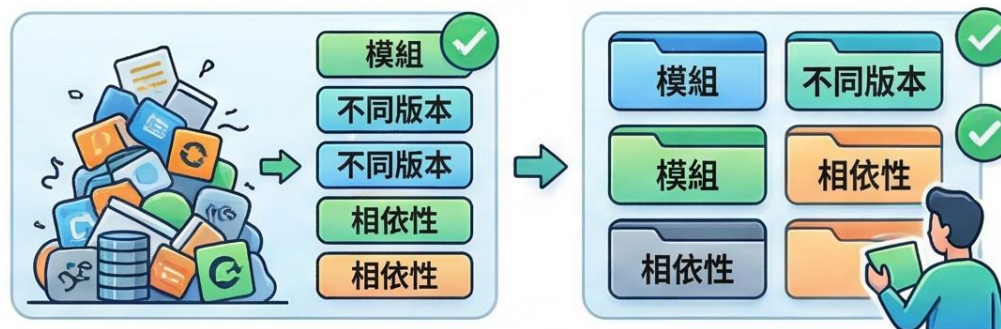


複雜軟體堆疊

多版本衝突

使用者困惑

2 Lmod 如何簡化



軟體模組化

輕鬆加載/卸載

3 Lmod 的核心功能



4 常用 Lmod 命令

module avail [名稱]
列出可用模組



module list
目前已加載

module load [模組]
加載模組

module unload [模組]
卸載模組

module keyword [關鍵字]
搜尋模組

5 Lmod 對使用者的好處



Forerunner1

```
[u8880716@ilgn01 ~]$ module avail

----- /pkg/modulefiles/software -----
apps/abaqus/2023
apps/abaqus/2024
apps/adf/2024.101-intelmpi-intel
apps/adf/2024.102-intelmpi-intel
apps/ansoft/v2023r2
apps/ansys/CFX/v2020r2
apps/ansys/CFX/v2022r2
apps/ansys/CFX/v2023r2
apps/ansys/CFX/v2024r1
apps/ansys/Fluent/v2020r2
apps/ansys/Fluent/v2022r2
apps/ansys/Fluent/v2023r2
apps/ansys/Fluent/v2024r1
apps/ansys/ICEMCFD/v2020r2
apps/ansys/ICEMCFD/v2022r2
apps/ansys/ICEMCFD/v2023r2
apps/ansys/ICEMCFD/v2024r1
apps/ansys/workbench_APDL/v2020r2
apps/ansys/workbench_APDL/v2022r2
apps/ansys/workbench_APDL/v2023r2
apps/ansys/workbench_APDL/v2024r1
apps/biovia/ms24
apps/biovia/ms25
apps/chem/castep-24.1
apps/chem/gromacs/2023.4
apps/chem/gromacs/2023.4/fftw/3.3.10
apps/chem/gromacs/2023.4/hwloc
apps/chem/gromacs/2023.4/openmpi/5.0.1
apps/chem/gromacs/2023.4/ucc/1.3.x
apps/chem/gromacs/2023.4/ucx/1.16
apps/cmp/BigDFT/1.9.5.25
apps/gaussian/g16
apps/lammps/lammps-2AUG2023-SHIK-potential
apps/lammps/lammps-2AUG2023
apps/lsdyna/R12.0.0
apps/lsdyna/R13.0.0
apps/lsdyna/R13.1.0
apps/matlab/R2023a
apps/matlab/R2024b
apps/molpro/2024.1
apps/openfoam/OpenFOAM-v2312-x86_64-Intel_OneAPI_AVX512
apps/openfoam/OpenFOAM-v2506-ARM64-GCC
apps/openfoam/OpenFOAM-v2506-x86_64-Intel_OneAPI_AVX512
apps/opensees/3.4.0-gcc
apps/opensees/3.4.0-icc
apps/opensees/3.5.0-gcc
apps/opensees/3.6.0-icc
apps/opensees/3.7.1-gcc
apps/opensees/3.7.1-icc
apps/orca/6.0.1
apps/qchem/6.2.0
apps/starccmp/STAR-CCM_Plus_v2506
libs/eigen/3.4.0
libs/hdf5/1.14.3
libs/scalapack/2.2.0-intel-oneapi-2023.2
tools/code-server
tools/jupyterlab
tools/miniconda3
tools/paraview/6.0.1
tools/paraview/client/5.12.0
tools/paraview/server/5.12.0
tools/rstudio/4.4.0
tools/singularity-ce/4.1.1

----- /pkg/modulefiles/middleware/compiler -----
gcc/8.5.0 (D) gcc/11.2.0 intel/2020_u4 intel/2022_3_1 intel/2024_01_46 (D)
gcc/10.4.0 intel/2019_u5 intel/2021_4 intel/2023_2
```

Lmod command

查看可安裝的軟體 module avail

查看軟體版本 module spider <package>

載入軟體 module load <package>

查看已載入軟體 module list

清除軟體 module purge

SINGULARITY 容器的核心功能與優勢

專為高效能運算 (HPC) 環境設計的容器技術

環境封裝



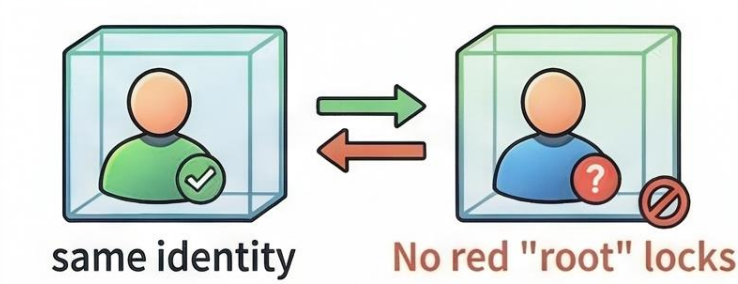
打包完整運行環境，確保跨系統
執行一致性

跨平台可攜



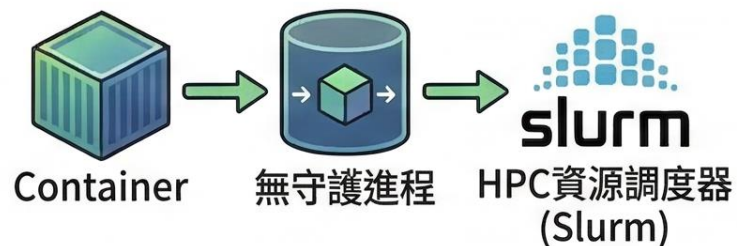
在個人電腦建立，部署至大規模叢集
或雲端，結果一致

權限一致



用戶身份與宿主系統相同，無需
root 權限，降低風險

輕量化部署



無守護進程，良好整合 HPC 資源調
度器，順利提交任務。

MPI 與 GPU 支援



綁定主機硬體資源（高速網絡、MPI、
GPU），效能接近裸機。

再現性



確保環境與軟體版本固定，實驗結果
更具再現性，利於學術交流

執行Singularity容器

1. 從Docker hub下載映像檔

□ `singularity pull pytorch_24.05-py3.sif docker://nvcr.io/nvidia/pytorch:24.05-py3`

2. 運行容器

□ 背景執行 `singularity exec/run --nv <image.sif> <command>`

□ 互動執行 `singularity run/shell --nv <image.sif>`

```
$ singularity shell instance:///instance3
```

```
Singularity>
```

□ `--nv` 允許容器使用GPU

□ 引用/work目錄 `singularity exec/run -B /work:/work --nv <image.sif> <command>`

- 已預設綁定home目錄，不用設定 `-B /home:/home`

□ 列出容器內的python套件

- `singularity exec/run --nv <image.sif> pip list`

建立客製化容器映像檔

1. 建立客製化容器映像檔需要sudo權限
2. 另尋有sudo權限Linux電腦 (例如TWCC VM, WSL, VirtualBox)建立
客製化容器映像檔
3. 安裝相關依賴庫、Go語言與Singularity
 - 注意Go語言與Singularity版本有搭配
4. 實作版本
 - Linux Ubuntu 24.04
 - Docker 28.3.2 and Nvidia container toolkit 1.17.8 (非必要)
 - Go 1.23.6
 - Singularity: Apptainer 1.4.1
 - <https://github.com/apptainer/apptainer/blob/main/INSTALL.md>

Install Go & Singularity

```
# Ensure repositories are up-to-date
sudo apt-get update
# Install debian packages for dependencies
sudo apt-get install -y \
    build-essential \
    libseccomp-dev \
    uidmap \
    fakeroot \
    cryptsetup \
    tzdata \
    dh-apparmor \
    libsubid-dev \
    pkg-config \
    curl wget git
```

```
# Install GO
export GOVERSION=1.23.6 OS=linux ARCH=amd64 # change this as you need
```

```
wget -O /tmp/go${GOVERSION}.${OS}-${ARCH}.tar.gz \
    https://dl.google.com/go/go${GOVERSION}.${OS}-${ARCH}.tar.gz
```

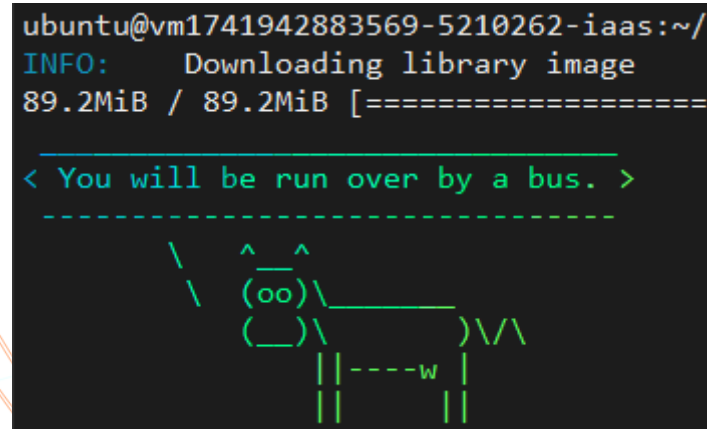
```
sudo tar -C /usr/local -xzf /tmp/go${GOVERSION}.${OS}-${ARCH}.tar.gz
```

```
# add /usr/local/go/bin to the PATH environment variable
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
source ~/.bashrc
```

```
# Install singulairty
git clone https://github.com/apptainer/apptainer.git
cd apptainer
git checkout v1.4.1
```

```
# Compiling Singularity
./mconfig
cd $(/bin/pwd)/builddir
make -j 4
sudo make install
```

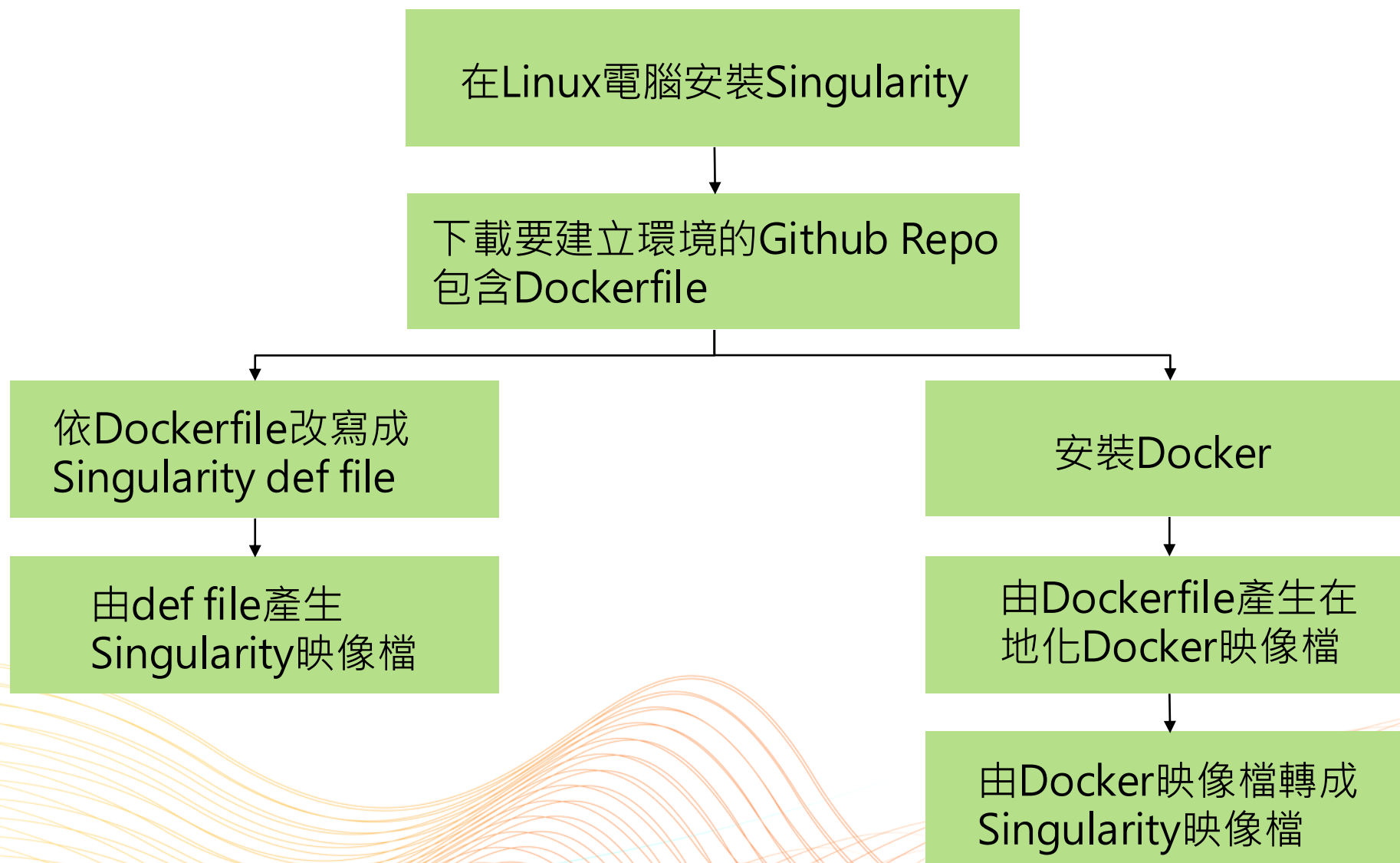
```
singularity --version
singularity run library://godlovedc/funny/lolcow
```



```
ubuntu@vm1741942883569-5210262-iaas:~/
INFO:   Downloading library image
89.2MiB / 89.2MiB [=====]

< You will be run over by a bus. >
-----
      ^__^
      (oo)\_______
      (_____)       )\\
      ||----w |
      ||     ||
```

建立客製化容器映像檔的步驟



建立定義檔Singularity Definition file

vi <definition>.def

```
Bootstrap: library
From: ubuntu:18.04
Stage: build

%setup
    touch /file1
    touch ${SINGULARITY_ROOTFS}/file2

%files
    /file1
    /file1 /opt

%environment
    export LISTEN_PORT=12345
    export LC_ALL=C

%post
    apt-get update && apt-get install -y netcat
    NOW=`date`
    echo "export NOW=\"${NOW}\"" >> $SINGULARITY_ENVIRONMENT

%runscript
    echo "Container was created $NOW"
    echo "Arguments received: $*"
    exec echo "$@"
```

從Docker hub取得映像檔

建立主機端和容器的檔案、資料夾

複製主機端的檔案、資料夾到容器

設定環境變數

執行安裝軟體指令

啟動容器時，執行程式

https://docs.sylabs.io/guides/3.7/user-guide/definition_files.html
HowTo : 建立 TWNIA2 容器

範例：Tensorflow for H100

Bootstrap: docker

From: nvcr.io/nvidia/tensorflow:23.10-tf2-py3

%help

此容器用於 CosmoFlow 資料預處理，解決了：

1. Protobuf 版本不相容問題（固定於 3.20.3）
2. H100 GPU (Compute 9.0) 支援
3. MPI 與 UCX 通訊優化

%post

更新系統並安裝必要工具

```
apt-get update && apt-get install -y --no-install-recommends \  
    build-essential \  
    libopenmpi-dev \  
    git
```

強制降級 Protobuf 以符合舊版 TensorFlow 邏輯，避免 TypeError

```
pip install --no-cache-dir --upgrade pip  
pip install --no-cache-dir "protobuf<3.21,>=3.20.3"
```

安裝並行計算與資料處理必要套件

```
pip install --no-cache-dir mpi4py numpy
```

%environment

解決 UCX 速度辨識錯誤的環境變數預設值

```
export UCX_TLS=tcp,self,sm
```

```
export UCX_IB_RETRY_COUNT=20
```

確保 Protobuf 使用 C++ 實作以獲得最佳效能

```
export PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=cpp
```

%runscript

```
exec python "$@"
```

建立映像檔

方法一：By singularity definition file

```
sudo singularity build <image>.sif <definition>.def
```

方法二：By docker local image

1. `docker build -t <repository>:<tag> .`
2. `sudo singularity build <image>.sif docker-daemon://<repository>:<tag>`

方法三：By docker local image and compressed file

1. `docker build -t <repository>:<tag> .`
2. `docker save <repository>:<tag> -o <image>.tar`
3. `sudo singularity build <image>.sif docker-archive://$(pwd)/<name>.tar`

```
ubuntu@vm1692770172744-3864139-iaas:~$ docker image list
REPOSITORY          TAG          IMAGE ID      CREATED        SIZE
mlperf-nvidia       language_model 7b697e4dcb2b 24 hours ago   22.5GB
llama-factory        latest       a334a786d902 3 months ago   22.7GB
nvcr.io/nvidia/pytorch 24.01-py3    8470a68886ff 7 months ago   22GB
nvcr.io/nvidia/pytorch 23.04-py3    b4428941db4f 16 months ago 20.4GB
```

範例：由Dockerfile轉換Singularity image (*.sif)

On TWCC VM (製作映像檔的環境)

1. `cd <path/to/Dockerfile>`
2. `docker build -t <repository>:<tag> .`
3. `sudo singularity build <image>.sif docker-daemon://<repository>:<tag>`
4. 傳送映像檔<image>.sif 到晶創25主機

```
ubuntu@vmcpuv-5372004-iaas:~$ cd bert_mlperf_3.0
ubuntu@vmcpuv-5372004-iaas:~/bert_mlperf_3.0$ ls
a30-run_and_time.sh  Dockerfile
a30.sub              extract_features.py
bmm1.py              file_utils.py
```

```
ubuntu@vmcpuv-5372004-iaas:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
bert_mlperf   pytorch-2304  dd6dc34e6fc8  33 minutes ago  22.5GB
ubuntu@vmcpuv-5372004-iaas:~$ ls
bert  bert_mlperf_3.0  bert_mlperf_pytorch_2304.sif  cosmoflow_tf.def
```

On Nano5

1. 在Slurm script引用Singularity 映像檔

```
#!/bin/bash
#SBATCH -A GOV108032      # iService Project id GOV112012 GOV110171
#SBATCH -J H100x8_bs16   # job name
#SBATCH -p normal        # partition
#SBATCH --ntasks-per-node=1 # Number of MPI tasks (i.e. processes)
#SBATCH --cpus-per-task=4  # Number of cores per MPI task
#SBATCH --nodes=1         # Maximum number of nodes to be allocated
#SBATCH --gres=gpu:8
#SBATCH -o %x_%j.out      # Path to the standard output file

export CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
export OMP_NUM_THREADS=1

SECONDS=0
SIF=/work/$(whoami)/sif/bert_mlperf_pytorch_2304.sif
SINGULARITY="singularity run -B /work:/work --nv $SIF"
```

安裝軟體

- 編譯語言：c/c++, fortran, CUDA
 - 透過Lmod引用gcc, openmpi, cmake, intel oneAPI
 - 編譯原始碼
- 直譯語言：python
 - 透過Lmod引用miniconda
 - pip install <package>

原始碼編譯

1. 在登入節點，編譯原始碼

1. 引用編譯器

```
module load gcc openmpi  
Module load intel
```

2. 製作makefile

```
mkdir build  
cd build  
cmake -DCMAKE_C_COMPILER=gcc -DCMAKE_CXX_COMPILER=g++ -DCMAKE_C_FLAGS="-g -O3" ..
```

3. 編譯makefile

```
make -j
```

2. 執行運算的腳本

```
#SBATCH ...  
  
module purge  
module gcc  
module load intel  
  
mpirun -n $SLURM_NTASKS <application> ...
```

Miniconda

1. 在登入節點，預先安裝環境

1. 引用Miniconda

```
module load miniconda3/24.11.1
```

2. 建立虛擬環境

```
conda create <env_name> python=3.x -y
```

3. 進入虛擬環境

```
conda activate <env_name>
```

4. 安裝套件

```
conda install <package>
```

```
pip install <package>
```

安裝套件路徑 ~/.conda/envs/<env_name>/lib/python3.x/site-packages/...

5. 退出虛擬環境

```
conda deactivate
```

2. 執行運算的腳本

```
#SBATCH ...

module purge
module load miniconda3/24.11.1
conda activate <env_name>

python train_script.py ...

conda deactivate
```

主機使用實作示範

1. 登入主機，密碼 + OTP驗證
2. 查詢資源
 1. 查看計劃錢包: `wallet <Project_ID>`
 2. 查看partition: `sinfo`
 3. 查看job狀態: `squeue`
 4. 查看HFS容量：

```
[user@ilgn01 ~]$ mmlsquota --block-size auto home1 work1
```

		Block Limits					File Limits					Remarks
Filesystem	type	blocks	quota	limit	in_doubt	grace	files	quota	limit	in_doubt	grace	
home1	USR	25.36G	1T	1.098T	637.6M	none	145361	0	0	98	none	
work1	USR	28.37G	200T	200.1T	0	none	10	0	0	0	none	

Case 1: mpi test

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main (int argc, char **argv) {
    int rc;
    int size;
    int myrank;

    rc = MPI_Init (&argc, &argv);
    if (rc != MPI_SUCCESS) {
        fprintf (stderr, "MPI_Init() failed");
        return EXIT_FAILURE;
    }

    rc = MPI_Comm_size (MPI_COMM_WORLD, &size);
    if (rc != MPI_SUCCESS) {
        fprintf (stderr, "MPI_Comm_size() failed");
        goto exit_with_error;
    }

    rc = MPI_Comm_rank (MPI_COMM_WORLD, &myrank);
    if (rc != MPI_SUCCESS) {
        fprintf (stderr, "MPI_Comm_rank() failed");
        goto exit_with_error;
    }
}
```

```
    fprintf (stdout, "Hello, I am rank %d/%d\n", myrank, size);

    MPI_Finalize();

    return EXIT_SUCCESS;

exit_with_error:
    MPI_Finalize();
    return EXIT_FAILURE;
}
```

mpitest.c

openmpi-4.1.6.def

Bootstrap: docker
From: ubuntu:24.04

sudo singularity build openmpi-4.1.6.sif openmpi-4.1.6.def

```
%files
    mpitest.c /opt

%environment
    export OMPI_DIR=/opt/mpi
    export SINGULARITY_OMPI_DIR=$OMPI_DIR
    export SINGULARITYENV_APPEND_PATH=$OMPI_DIR/bin
    export SINGULARITYENV_APPEND_LD_LIBRARY_PATH=$OMPI_DIR/lib

%post
    echo "Installing required packages..."
    apt-get update && apt-get install -y wget git bash gcc gfortran g++ make file bzip2

    echo "Installing Open MPI"
    export OMPI_DIR=/opt/mpi
    export OMPI_VERSION=4.1.6
    export OMPI_URL="https://download.open-mpi.org/release/open-mpi/v4.1/openmpi-$OMPI_VERSION.tar.bz2"
    mkdir -p /tmp/mpi
    mkdir -p /opt
    # Download
    cd /tmp/mpi && wget -O openmpi-$OMPI_VERSION.tar.bz2 $OMPI_URL && tar -xjf openmpi-$OMPI_VERSION.tar.bz2
    # Compile and install
    cd /tmp/mpi/openmpi-$OMPI_VERSION && ./configure --prefix=$OMPI_DIR && make install

    # Set env variables so we can compile our application
%environment
    export PATH=$OMPI_DIR/bin:/usr/bin:$PATH
    export LD_LIBRARY_PATH=$OMPI_DIR/lib:$LD_LIBRARY_PATH
    export MANPATH=$OMPI_DIR/share/man:$MANPATH

%post
    #echo "Compiling the MPI application..."
    #cd /opt && mpicc -o mpitest mpitest.c
```

run.slurm

```
#!/bin/bash
#SBATCH --account=<project id>          # (-A) iService Project ID
#SBATCH --job-name=mpi                  # (-J) Job name
#SBATCH --partition=development         # (-p) Slurm partition
#SBATCH --nodes=2                      # (-N) Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=1               # (-c) Number of cores per MPI task
#SBATCH --ntasks-per-node=2            # Maximum number of tasks on each node
#SBATCH --time=00:30:00                 # (-t) Wall time limit (days-hrs:min:sec)

export PMIX_MCA_gds=^ds12
export PMIX_MCA_psec=^munge

SIF="/work1/u8880716/sif/openmpi-4.1.6.sif"
SINGULARITY="singularity run $SIF"

COMPILIE="srun $SINGULARITY mpicc -o mpitest mpitest.c"
echo "$COMPILIE";$COMPILIE

RUN="srun --mpi=pmix -n 4 $SINGULARITY ./mpitest"
echo "$RUN" ;$RUN
```

run.slurm

sbatch run.slurm

```
[u8880716@ilgn01 demo]$ sbatch run.slurm
Submitted batch job 281267
[u8880716@ilgn01 demo]$ cat *281267*
srun singularity run /work1/u8880716/sif/openmpi-4.1.6.sif mpicc -o mpitest mpitest.c
srun --mpi=pmix -n 4 singularity run /work1/u8880716/sif/openmpi-4.1.6.sif ./mpitest
Hello, I am rank 0/4
Hello, I am rank 3/4
Hello, I am rank 1/4
Hello, I am rank 2/4
```

miniWeather

via X86 vs ARM

使用創進一號主機計算資源工具

遠端(Remote)

登入節點

檔案傳輸節點

互動式繪圖節點

OnDemand

X86 f1-ilgn01.nchc.org.tw
f1-ilgn02.nchc.org.tw

f1-dtn01.nchc.org.tw
f1-dtn02.nchc.org.tw

f1-intgpn01.nchc.org.tw
f1-intgpn02.nchc.org.tw

f1-stn01.nchc.org.tw
f1-stn02.nchc.org.tw

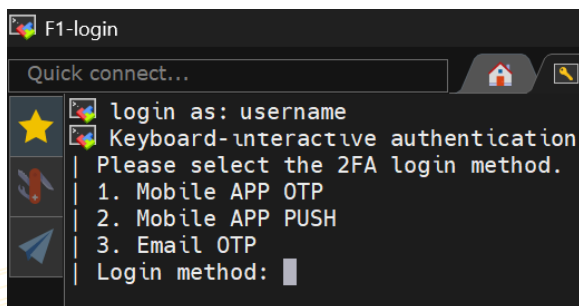
ARM f1-nlgn01.nchc.org.tw
f1-nlgn02.nchc.org.tw

連接埠 22

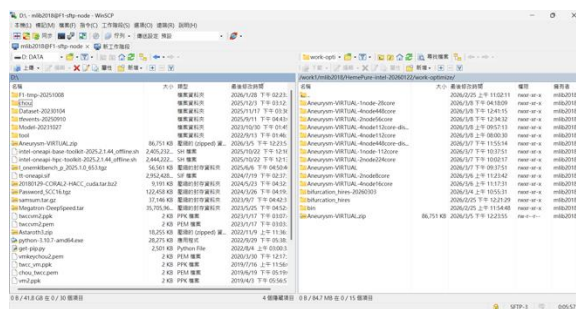
22

1111

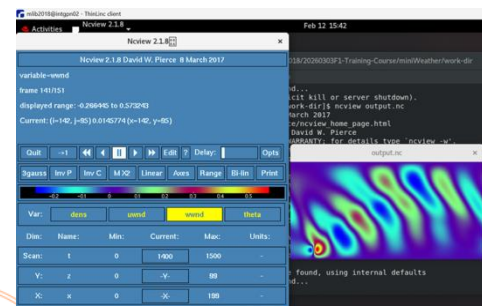
地端(Local) 便利順手的工具
例如：



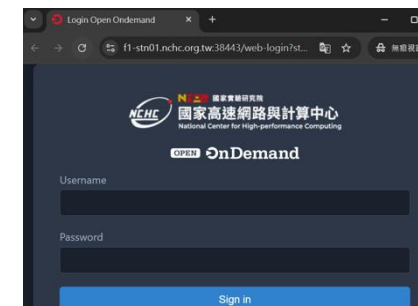
MobaXterm



WinSCP

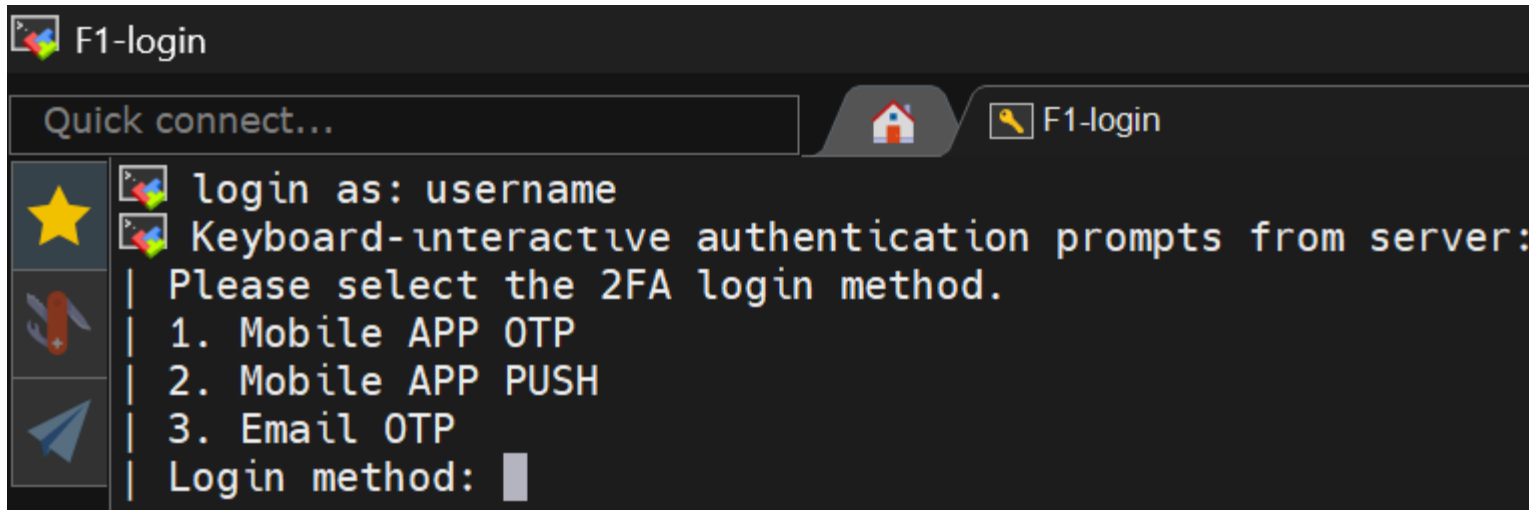


ThinLinc



Chrome

遠端登入「指令輸入」使用模式



1



2



檔案傳輸 via WinSCP

D:\ - mlib2018@F1-sftp-node - WinSCP

本機(L) 標記(M) 檔案(F) 指令(C) 工作階段(S) 選項(O) 遠端(R) 說明(H)

同步 上傳 下載 傳送設定 預設

mlib2018@F1-sftp-node x 新工作階段

D: DATA

上傳 編輯 屬性 新增

遠端 Remote

work-opti 尋找檔案 下載 編輯 屬性 新增

Local 地端			
名稱	大小	類型	最後修改時間
F1-tmp-20251008		檔案資料夾	2026/1/28 下午 02:23:
chou		檔案資料夾	2025/12/3 下午 03:12:
Dataset-20230104		檔案資料夾	2025/11/17 下午 03:36:
tfevents-20250910		檔案資料夾	2025/9/11 下午 04:43:
Model-20231027		檔案資料夾	2023/10/30 下午 01:45:
tool		檔案資料夾	2022/9/13 下午 01:46:
Aneurysm-VIRTUAL.zip	86,751 KB	壓縮的 (zipped) 資...	2026/3/5 下午 12:23:5
intel-oneapi-base-toolkit-2025.2.1.44_offline.sh	2,405,232...	SH 檔案	2025/10/22 下午 12:16:
intel-oneapi-hpc-toolkit-2025.2.1.44_offline.sh	2,444,222...	SH 檔案	2025/10/22 下午 12:13:
l_onemklbench_p_2025.1.0_653.tgz	56,561 KB	壓縮的封存資料夾	2025/6/6 下午 04:50:4
tt-oneapi.sif	2,952,428...	SIF 檔案	2024/7/19 下午 02:37:
20180129-CORAL2-HACC_cuda.tar.bz2	9,191 KB	壓縮的封存資料夾	2024/5/23 下午 04:32:
Password_SCC16.tgz	122,458 KB	壓縮的封存資料夾	2024/3/26 下午 04:19:
samsum.tar.gz	37,146 KB	壓縮的封存資料夾	2023/9/7 下午 04:42:3
Megatron-DeepSpeed.tar	35,705,96...	壓縮的封存資料夾	2023/5/25 下午 04:52:
twccvm2.ppk	2 KB	PPK 檔案	2023/1/17 下午 03:07:
twccvm2.pem	2 KB	PEM 檔案	2023/1/17 下午 03:03:
Astaroth3.zip	18,255 KB	壓縮的 (zipped) 資...	2022/11/9 上午 11:36:
python-3.10.7-amd64.exe	28,275 KB	應用程式	2022/9/29 下午 05:38:
get-pip.py	2,501 KB	Python File	2022/8/4 上午 03:00:3
vmkeychou2.pem	2 KB	PEM 檔案	2020/3/30 下午 12:17:
twcc_vm.ppk	2 KB	PPK 檔案	2019/7/16 上午 11:56:
chou_twcc.pem	2 KB	PEM 檔案	2019/6/19 下午 05:19:
vm2.ppk	2 KB	PPK 檔案	2019/4/3 下午 05:56:

0 B / 41.8 GB 在 0 / 30 個項目

4 個隱藏項目

/work1/mlib2018/HemePure-intel-20260122/work-optimize/				
名稱	大小	最後修改時間	權限	擁有者
..		2026/2/25 上午 11:02:11	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-1node-28core		2026/3/8 下午 04:18:09	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-4node448core		2026/3/8 下午 12:41:15	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-2node56core		2026/3/8 下午 12:34:32	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-4node112core-dis...		2026/3/8 上午 09:57:13	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-4node112core		2026/3/8 上午 08:00:30	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-4node448core-dis...		2026/3/7 下午 11:55:14	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-1node-112core		2026/3/7 下午 10:37:51	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-2node224core		2026/3/7 下午 10:02:17	rw-r--r--	mlib2018
Aneurysm-VIRTUAL		2026/3/7 下午 09:37:51	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-2node8core		2026/3/6 上午 11:23:42	rw-r--r--	mlib2018
Aneurysm-VIRTUAL-4node16core		2026/3/6 上午 11:17:31	rw-r--r--	mlib2018
bifurcation_hires-20260303		2026/3/4 上午 10:55:31	rw-r--r--	mlib2018
bifurcation_hires		2026/2/25 下午 12:21:29	rw-r--r--	mlib2018
bin		2026/2/25 上午 11:54:48	rw-r--r--	mlib2018
Aneurysm-VIRTUAL.zip	86,751 KB	2026/3/5 下午 12:23:55	rw-r--r--	mlib2018

0 B / 84.7 MB 在 0 / 15 個項目

SFTP-3 0:05:57

互動式繪圖節點 via ThinLinc Client (1/3)

ThinLinc client

ThinLinc Version 4.18.0 Build 3768

Server: f1-intgpn02.nchc.org.tw

Username: username

Password:

☐ End existing session

Options...

Exit Advanced<< Connect

Enter username and password to connect.

1st

5th

ThinLinc client options

Display Local devices Optimization Security Advanced

SSH port

☐ 22 (default SSH port)

☐ 80 (HTTP)

☒ 1111

2nd

Authentication method

☒ Password

☐ Public key

☐ Smart card

☐ Kerberos ticket

Details...

Cancel OK

ThinLinc Client Options

Display Local Devices Optimization Security Advanced

Start a Program

☐ Start the following program

Command: tl-single-app firefox

Session Options

☐ Enable shadowing

Popup menu key F8

☐ Emulate middle mouse button

☒ Send system keys

Reconnect policy

☐ Automatically reconnect to a disconnected session

☒ Always ask how multiple sessions should be handled

3th

Software Updates

☒ Check for client updates

Cancel OK

4th

Authentication

?

OTP: 020978

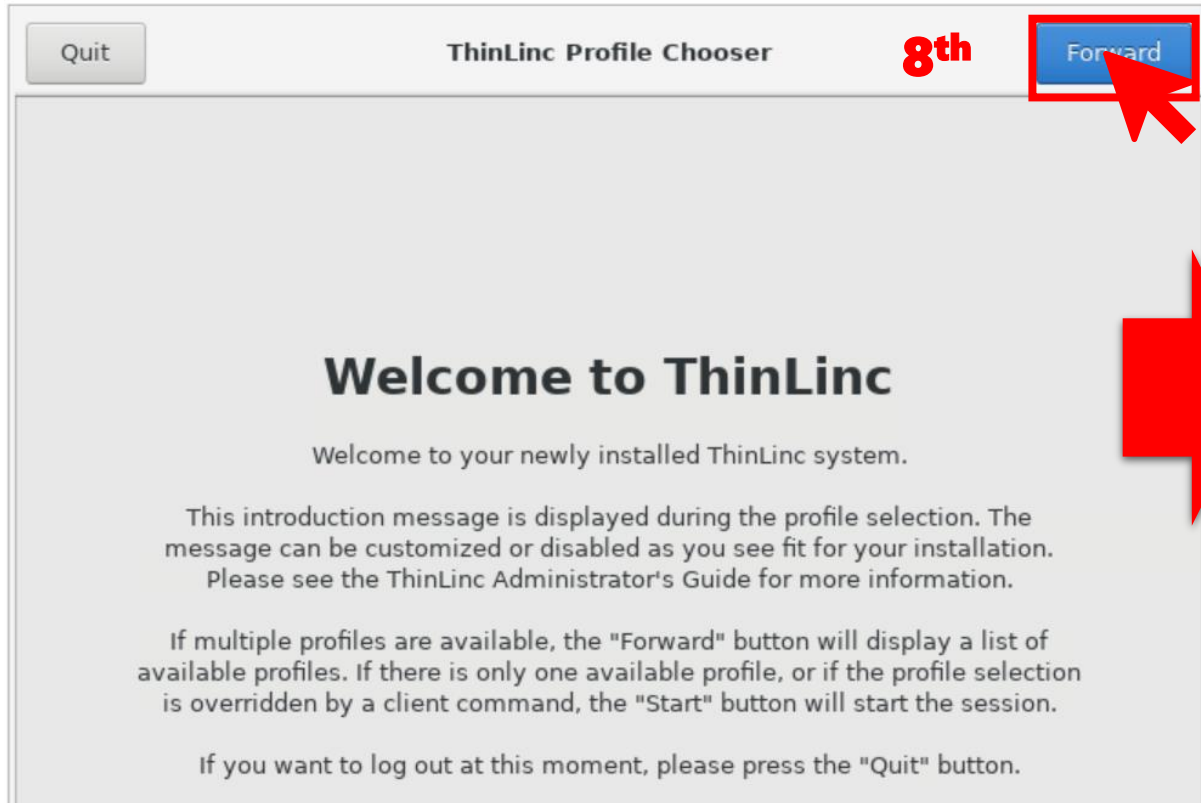
6th

.....

OK Cancel

7th

互動式繪圖節點 via ThinLinc Client (2/3)



10th

🔍 Type to search...



@intgpn02:~



File Edit View Search Terminal Help

@intgpn02 ~]\$



X86 架構 CPU 計算環境

以下佇列每一核心配置4.3GB(即4308MB)記憶體

(表格更新日期為 2025/11/27)

Slurm 佇列 (Partition)

X86 架構 CPU 計算...

ARM 架構 CPU 計算...

實作範例教學

全部展開

回到頂部

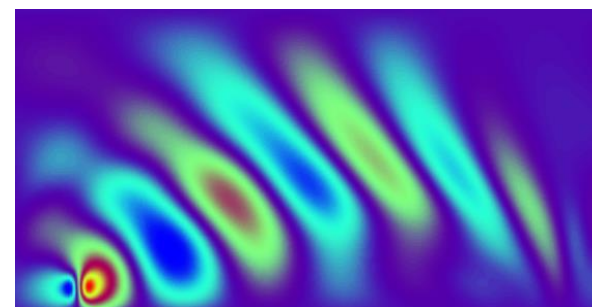
移至底部



佇列名稱	可用 核心數範圍	最長 可執行時間 (hour)	每位 用戶 可同時 執行 工作數	每位 用戶 可派送 工作數	計算工作派送節點
development	1~1120	8	1	2	限制從登入節點、 互動式繪圖節點派送， 即 ilgcn[01-02] 與 intgpn[01-02]
ct112	1~112	96	28	32	
ct448	113~448	96	24	32	
ct1k	449~1120	64	6	9	
ct2k	1121~2240	48	4	6	
ct4k	2241~4480	48	1	3	
ct8k	4481~8960	48	1	2	

miniWeather

- miniWeather 是一個模擬大氣天氣與氣候基本動力學的小型數值模式。它主要用於展示並行程式設計技術，並適用於多種運算平台，如 CPU 和 GPU。該程式透過簡化的氣象模型，使使用者能夠專注於高效能計算和並行計算技術的應用。
- **物理現象**：miniWeather 模擬乾燥、可壓縮的分層非靜力流。
- **高效能運算支援**：
 - 可運行於多種架構，包括 CPU、GPU 以及高效能運算集群（HPC）。
 - 透過並行程式設計提升計算效率。
 - 支援 **MPI** / OpenMP / OpenACC / Fortran
- **應用領域**
 - 氣象模擬：研究小尺度大氣現象的行為。
 - 高效能計算（HPC）訓練：示範如何利用 MPI/OpenMP 進行並行程式開發。
 - GPU 運算



Mountain
Gravity Waves

<https://github.com/mrnorman/miniWeather>

程式安裝 (1/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體

✓ parallel-netcdf 軟體

<https://parallel-netcdf.github.io/wiki/Download.html>

✓ 下載 parallel-netcdf 軟體

\$ wget <https://parallel-netcdf.github.io/Release/pnetcdf-1.14.1.tar.gz>

\$ tar -zxvf pnetcdf-1.14.1.tar.gz

\$ cd pnetcdf-1.14.1

\$./configure --prefix=\$HOME/opt

\$ make -j8

\$ make install

```
$ ./configure --prefix=$HOME/opt
PnetCDF Version 1.14.1

Features:  Build static libraries      - yes
           Build shared libraries       - yes
           Build Fortran APIs           - yes
           Build C++ APIs               - yes

Compilers: MPICC      = /pkg/compiler/intel/2024/mpi/2021.11/bin/mpicc
           MPICXX     = /pkg/compiler/intel/2024/mpi/2021.11/bin/mpicxx
           MPIF77     = /pkg/compiler/intel/2024/mpi/2021.11/bin/mpif77
           MPIF90     = /pkg/compiler/intel/2024/mpi/2021.11/bin/mpif90
           CFLAGS     = -g -O2
           CXXFLAGS   = -g -O2
           FFLAGS     = -g -O2
           FCFLAGS    = -g -O2
           SEQ_CC     = /usr/bin/gcc
```

成功

程式安裝 (2/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體 在 \$HOME/opt 成功

② 安裝 miniWeather 軟體

✓ 下載 miniWeather 軟體

```
$ cd $HOME
```

```
$ git clone https://github.com/mrnorman/miniWeather.git
```

```
$ cd miniWeather
```

```
$ git submodule update --init --recursive
```

✓ 編譯

```
$ cd c
```

```
$ mkdir build-F1
```

```
$ cd build-F1
```

```
$ cp ../build/cmake_clean.sh .
```

程式安裝 (3/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體 在 \$HOME/opt 成功

② 安裝 miniWeather 軟體

✓ 編譯 (Cont.)

建立 cmake_intel2024.sh

```
#!/bin/bash
module load intel/2024_01_46
export TEST_MPI_COMMAND="mpirun -n 1"
unset CUDAFLAGS
unset CXXFLAGS
PARALLEL_NETCDF_ROOT=$HOME/opt

./cmake_clean.sh

cmake -DCMAKE_CXX_COMPILER=mpicxx \
      -DCXXFLAGS="-Ofast -march=native -mtune=native -DNO_INFORM -std=c++11 -I${PARALLEL_NETCDF_ROOT}/include" \
      -DLDFLAGS="-L${PARALLEL_NETCDF_ROOT}/lib -lpnetcdf" \
      -DOPENMP_FLAGS="-fopenmp" \
      -DNX=200      #Xgrid \
      -DNZ=100 \
      -DDATA_SPEC="DATA_SPEC_GRAVITY_WAVES" 模擬問題 \
      -DSIM_TIME=1500 \
      ..
```

程式安裝 (4/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體 在 \$HOME/opt 成功

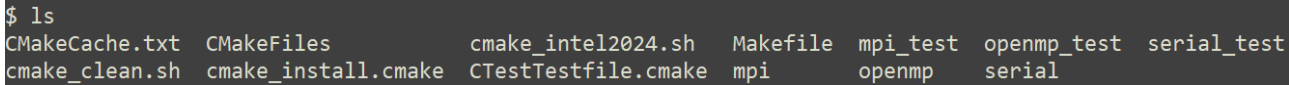
② 安裝 miniWeather 軟體

✓ 編譯 (Cont.)

```
$ bash cmake_intel2024.sh
```

```
$ make
```

```
$ ls
```



```
CMakeCache.txt  CMakeFiles      cmake_intel2024.sh  Makefile  mpi_test  openmp_test  serial_test  
cmake_clean.sh  cmake_install.cmake  CTestTestfile.cmake  mpi      openmp    serial
```

✓ 建立工作目錄

```
$ cd $HOME/miniWeather
```

```
$ mkdir work-dir
```

```
$ cd work-dir
```

```
$ cp ../c/build-F1/mpi .
```

程式安裝 (5/6)

➤ 計算環境 \$ module load intel/2024_01_46

① 安裝parallel-netcdf 軟體 在 \$HOME/opt 成功

② 安裝 miniWeather 軟體

③ 執行 2node-4MPI

✓ 建立批次作業檔案 job-2node-4mpi

```
#!/bin/bash
#SBATCH --account=project_id      # (-A) iService Project ID
#SBATCH --job-name=p2node4mpi    # (-J) Job name
#SBATCH --partition=development  # (-p) Slurm partition development ct112
#SBATCH --nodes=2                # (-N) Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=1        # (-c) Number of cores per MPI task
#SBATCH --ntasks-per-node=2      # Maximum number of tasks on each node
#SBATCH --output=zzz-2node-4MPI-%j.out # outputfile
export OMP_NUM_THREADS=1
export LD_LIBRARY_PATH=$HOME/opt/lib:$LD_LIBRARY_PATH
export PATH=$HOME/opt/bin:$PATH
ml intel/2024_01_46
srun --mpi=pmix_v4 ./mpi
```

\$ batch job-2node-4mpi

安裝 ncview 繪圖軟體

- ① 建立新工作環境 ncview_env，並且安裝 ncview

```
$ ml tools/miniconda3
```

```
$ conda create -n ncview_env -c conda-forge ncview -y
```

```
#  
# To activate this environment, use  
#  
#   $ conda activate ncview_env  
#  
# To deactivate an active environment, use  
#  
#   $ conda deactivate
```

登入互動繪圖節點後，進入 ncview_env 工作環境

```
$ ml tools/miniconda3
```

```
$ conda activate ncview_env
```

```
(ncview_env) $ ncview output.nc
```

Activities Ncview 2.1.8

Ncview 2.1.8

Ncview 2.1.8 David W. Pierce 8 March 2017

variable=wwnd

frame 141/151

displayed range: -0.266445 to 0.573243

Current: (i=142, j=95) 0.0145774 (x=142, y=95)

Quit ->1 << < || > >> Edit ? Delay: Opts

3gauss Inv P Inv C M X2 Linear Axes Range Bi-lin Print

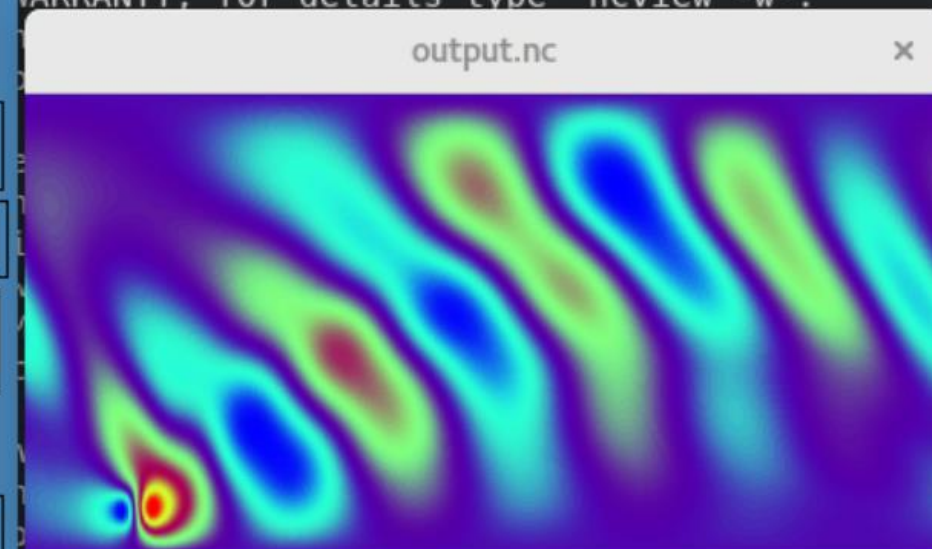
Var: dens uwnd **wwnd** theta

Dim:	Name:	Min:	Current:	Max:	Units:
Scan:	t	0	1400	1500	-
Y:	z	0	-Y-	99	-
X:	x	0	-X-	199	-

Feb 12 15:42

018/20260303F1-Training-Course/miniWeather/work-dir

```
nd...
icit kill or server shutdown).
work-dir]$ ncview output.nc
March 2017
ce/ncview_home_page.html
David W. Pierce
WARRANTY; for details type `ncview -w'.
```



```
e found, using internal defaults
nd...
```

ARM 實機操作

```
$ module load nvhpc/25.5
```

安裝類似X86

```
#!/bin/bash
#SBATCH --account=Project_id      # (-A) iService Project ID
#SBATCH --job-name=p4mpi          # (-J) Job name
#SBATCH --partition=arm-dev       # (-p) Slurm partition arm-dev arm144 arm576 arm1440 for ARM Cluster
#SBATCH --nodes=2                 # (-N) Maximum number of nodes to be allocated
#SBATCH --cpus-per-task=1         # (-c) Number of cores per MPI task
#SBATCH --ntasks-per-node=2       # Maximum number of tasks on each node
#SBATCH --output=zzz-2node-4MPI-%j.out # (-o) Path to the standard output file

ml nvhpc/25.5
export OMP_NUM_THREADS=1
export LD_LIBRARY_PATH=$HOME/opt/lib:$LD_LIBRARY_PATH
export PATH=$HOME/opt/bin:$PATH
mpirun -np $SLURM_NTASKS ./mpi
```

F1 X86 vs ARM

- x86 計算節點：552 台
 - ✓ 雙CPU架構，搭載 Intel Xeon Platinum 8480+ (56 核心 / 2.0GHz)
 - ✓ @Node has 112 cores
- ARM 計算節點：40 台
 - ✓ NVIDIA Grace CPU Superchip (72 核心 / 2.6GHz)
 - ✓ @台計算節點有 144 Arm Neoverse V2 cores

常用資訊

<https://man.twcc.ai/@f1-manual/manual>

- 計算資源異動
- 使用手冊
- 登入節點 (login node)、計算節點群 (partition)、檔案傳輸節點、互動式繪圖節點等

客服專線：0809-091-365

帳號與計畫諮詢信箱：iservice@niar.org.tw

技術諮詢信箱：isupport@niar.org.tw



客服專線：0809-091-365

帳號與計畫諮詢信箱：iservice@niar.org.tw

技術諮詢信箱：isupport@niar.org.tw